ASSIGNMENT 1: CLASSIFICATION

Lecturer: PhD. Le Thanh Sach

Group: Group6

TEAM MEMBER

1. Bui Nguyen Gia Huy - 2570201
2. Nguyen Ngoc Minh - 2570111
3. Pham Minh Quang – 2570302
4. Le Ngoc Minh Thu - 2570331

CONTENT

IMAGE CLASSIFICATION

TEXT CLASSIFICATION

MULTIMODAL CLASSIFICATION

IMAGE CLASSFICATION

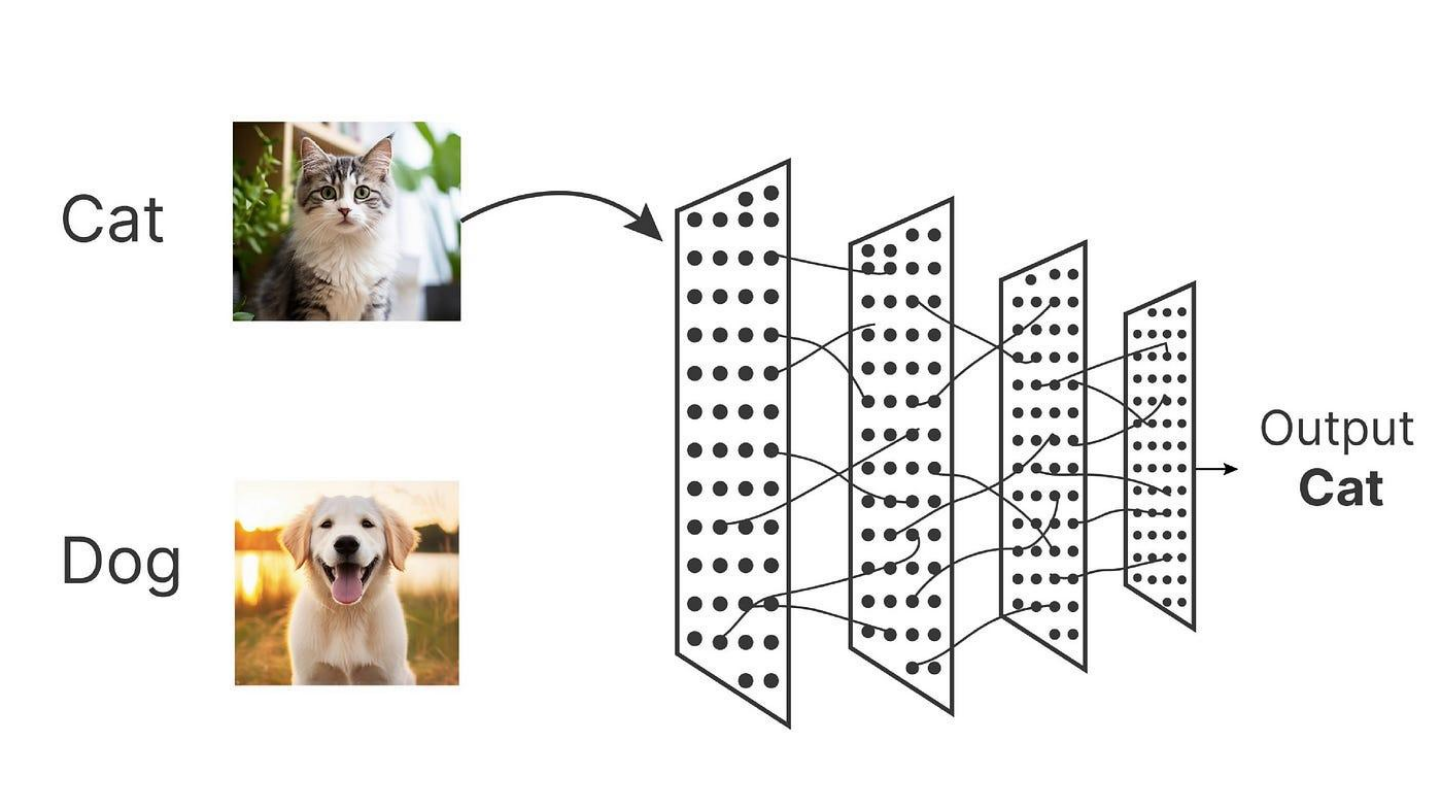
INTRODUCTION

Image Classification là bài toán dự đoán/gán nhãn cho mỗi hình ảnh dựa trên nội dung của nó.

Mục tiêu: tự động gán một nhãn danh mục duy nhất cho hình ảnh đầu vào dựa trên các đặc trưng thị giác đã học.

Ứng dụng phổ biến: nhận dạng vật thể, chẩn đoán y tế, giám sát an ninh, phân loại sản phẩm

INTRODUCTION



INTRODUCTION

Các metric đánh giá chính trong bài toán Image Classification:

- Accuracy (Độ chính xác)
- Precision (Độ chính xác của nhãn dương)
- Recall (Độ nhạy / Tỷ lệ phát hiện)
- F1-score (Điểm F1)
- Confusion Matrix (Ma trận nhầm lẫn)

INTRODUCTION

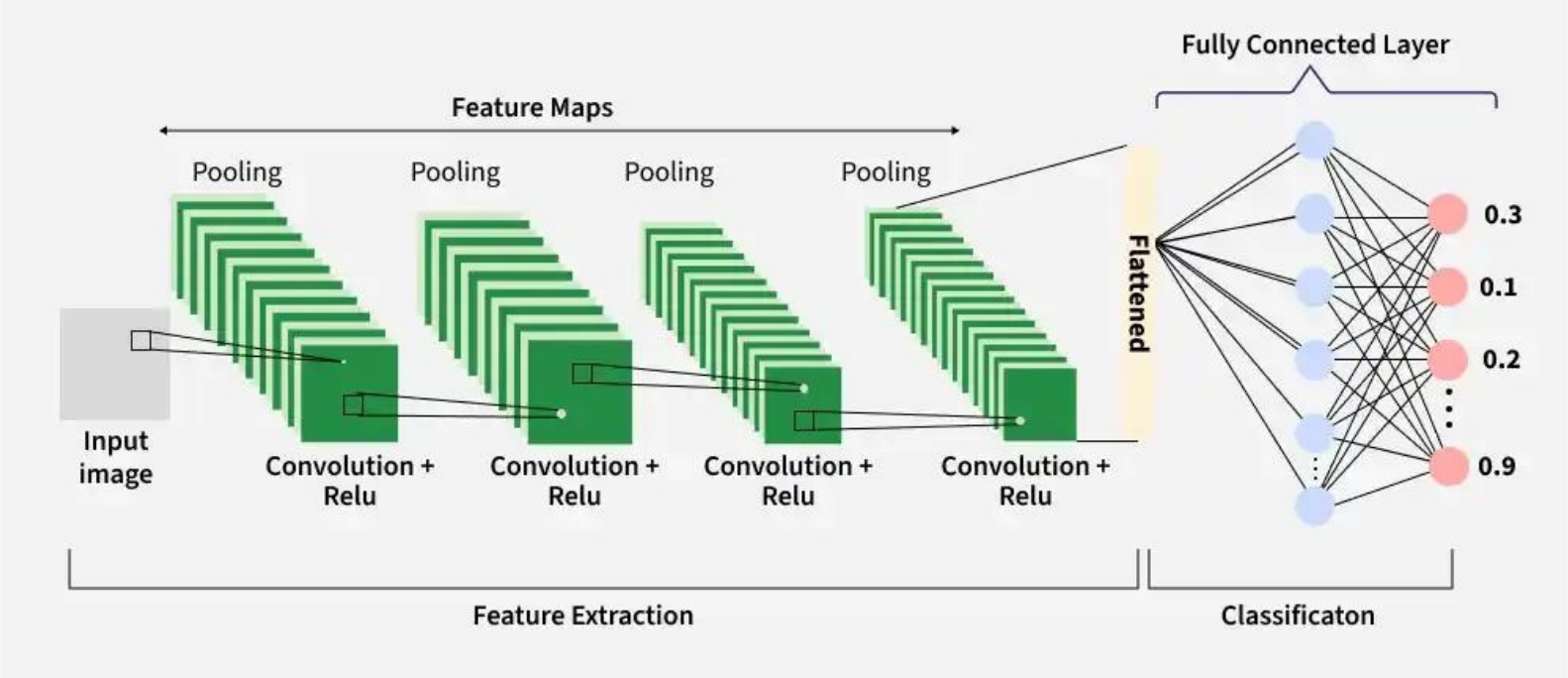
Các phương pháp cổ điển:

- Trích xuất đặc trưng (SIFT, HOG, SURF)
- Phân loại (SVM – Support Vector Machine, Random Forest, k-NN)
- Hạn chế: hiệu quả phụ thuộc vào kĩ năng thiết kế đặc trưng, khó thích nghi với tập dữ liệu lớn.

Các phương pháp hiện đại:

- CNNs (ResNet, VGG, Inception)
- Vision Transformer
- Ưu điểm: độ chính xác vượt trội, khả năng tổng quát hóa cực cao trên các tập dữ liệu khổng lồ.

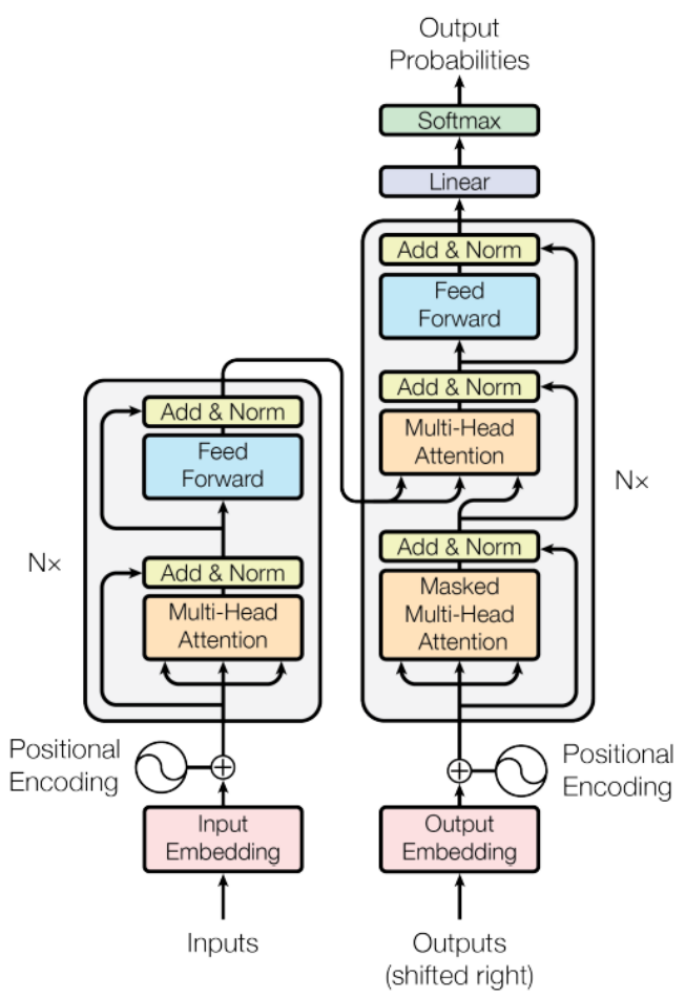
INTRODUCTION



CNN Architecture

(source: https://www.geeksforgeeks.org/machine-learning/introduction-convolution-neural-network/)

INTRODUCTION



Vision Transformer

(source: https://blog.roboflow.com/vision-transformers/)

INTRODUCTION

Lựa chọn so sánh cho 2 loại kiến trúc:

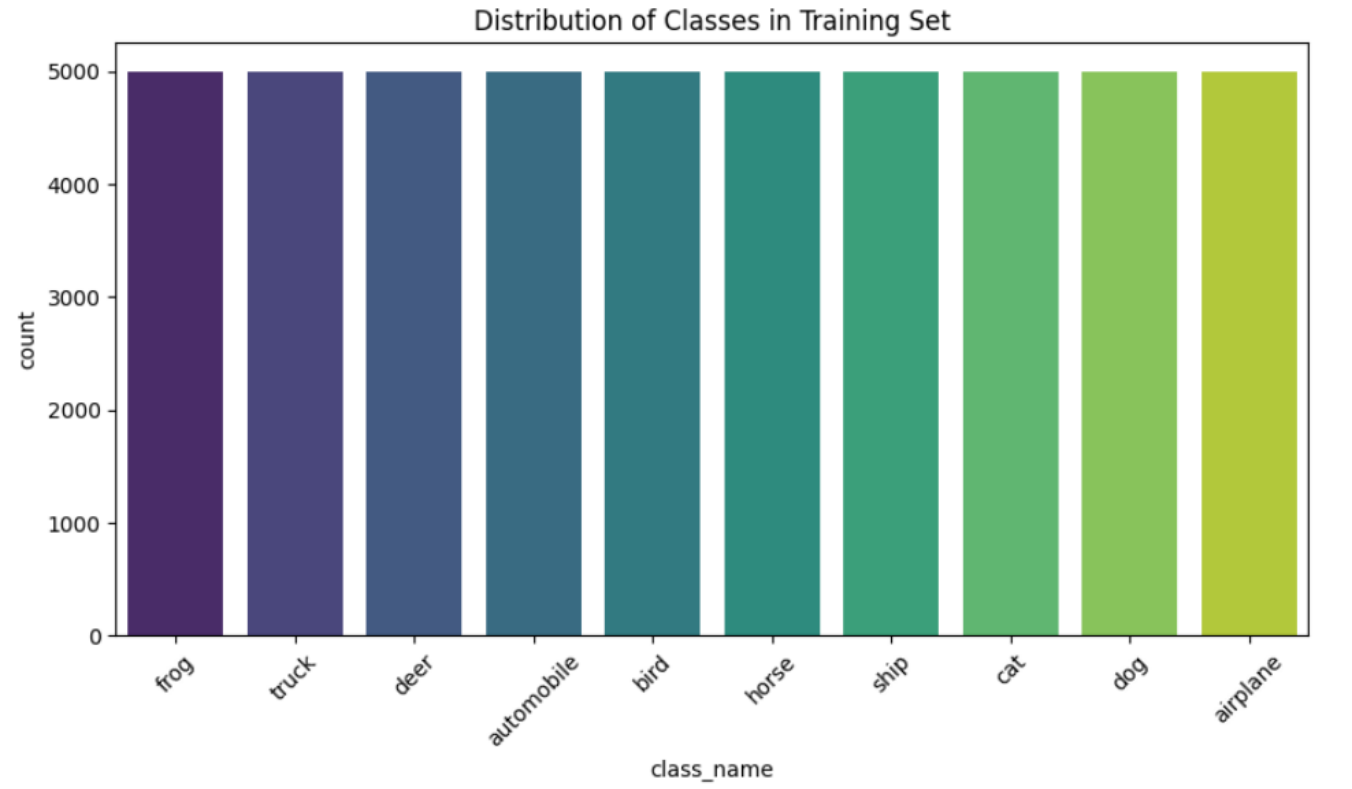
- **ResNet50** – CNN-based Model (He et al., 2016, arXiv:1512.00085)
- **ViT** – Transformer-based Model (Dosovitskiy et al., 2020, arXiv:2010.11929)

DATASET

CIFAR-10 Dataset:

- **Nguồn:** https://www.cs.toronto.edu/~kriz/cifar.html
- Bộ sưu tập các hình ảnh được sử dụng phổ biến trong các bài toán thị giác máy tính.
- **Quy mô:** 60.000 hình ảnh màu kích thước 32x32 pixel.
- **Phân loại:** 10 lớp đối tượng (máy bay, ô tô, chim, mèo, nai, ...).
- **Cấu trúc:** 50.000 ảnh huấn luyện và 10.000 ảnh kiểm thử.
- **Đặc điểm:** Mỗi lớp có chính xác 6.000 hình ảnh, tạo nên một tập dữ liệu cân bằng hoàn hảo.

EDA

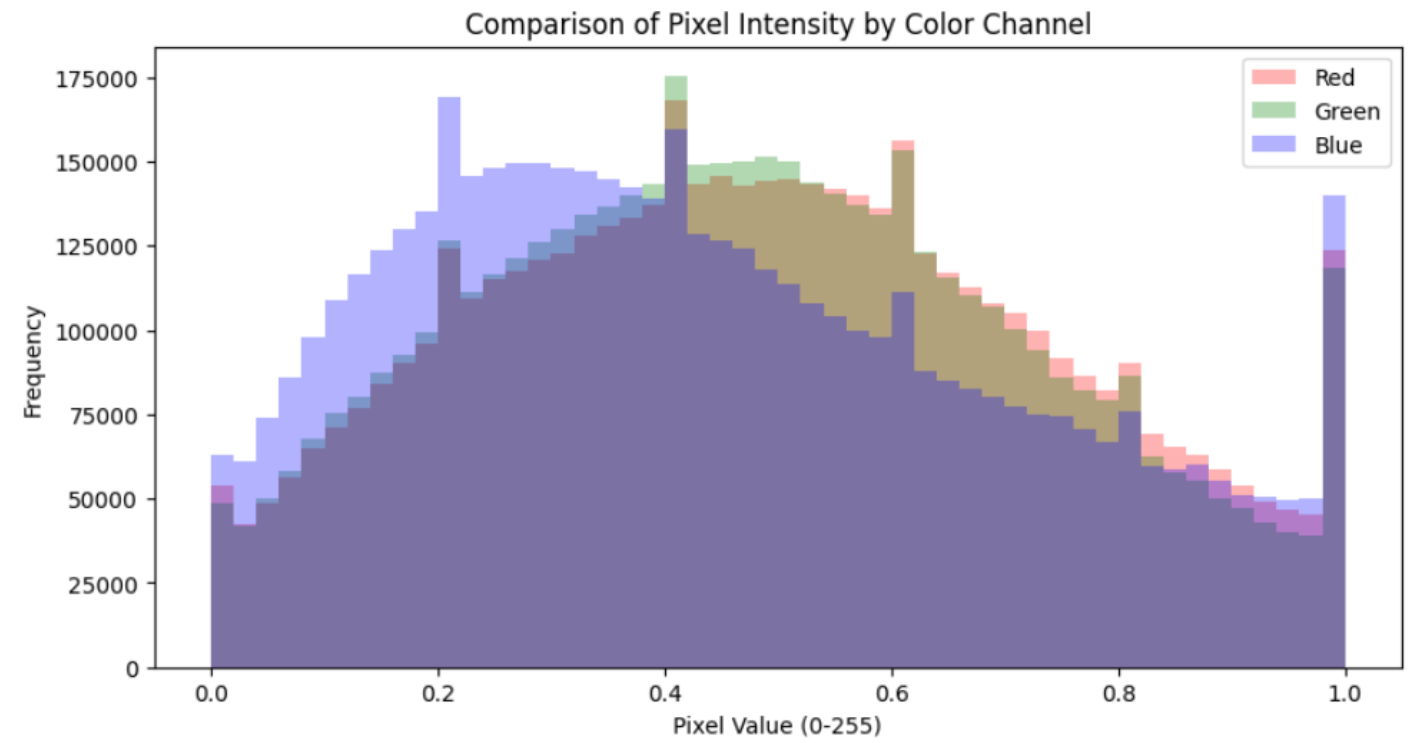


Class Distribution

EDA

Visual Integrity

EDA




```
Red Channel - Mean: 125.31, Std: 62.99
Green Channel - Mean: 122.95, Std: 62.09
Blue Channel - Mean: 113.87, Std: 66.70
```

Pixel Distribution

DATA PREPROCESSING

Quá trình tiền xử lý gồm 4 bước:

- Chuẩn hóa pixel về khoảng [0, 1]
- One-hot encoding nhãn
- Phân tách tập validation (80/20)
- Data Augmentation (Standard + CutMix)

CutMix là kỹ thuật quan trọng giúp mô hình học được tính tổng quát cao hơn.

DATA PREPROCESSING

Phân chia tập dataset:

- 10000 samples
- Tỷ lệ train – test: 80 – 20
- Kích thước ảnh: 32 x 32 x 3

```
# 3. Validation Split
# We set aside 10,000 images (20%) to tune hyperparameters
X_train, X_val, y_train_cat, y_val_cat = train_test_split(
    X_train, y_train_cat, test_size=0.2, random_state=42, stratify=y_train
)

print(f"Preprocessing Complete.")
print(f"Train: {X_train.shape}, Val: {X_val.shape}, Test: {X_test.shape}")
```

```
Preprocessing Complete.
Train: (40000, 32, 32, 3), Val: (10000, 32, 32, 3), Test: (10000, 32, 32, 3)
```

DATA PREPROCESSING

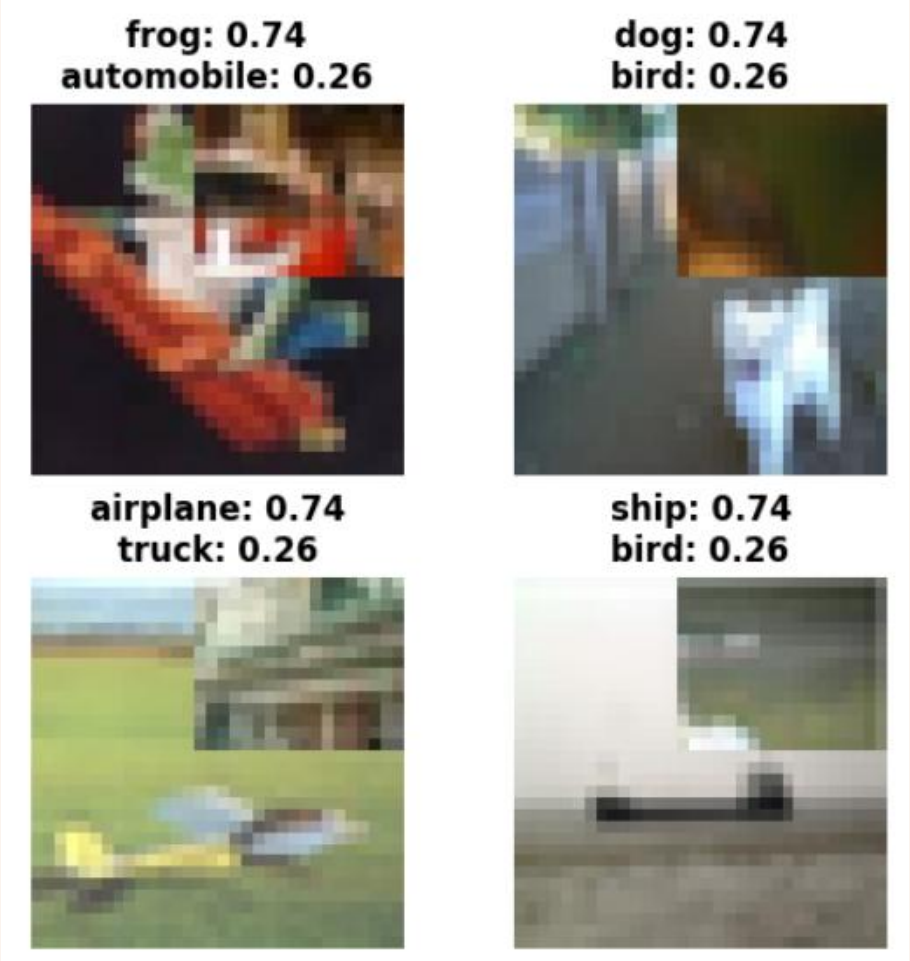
Tăng cường dữ liệu – Standard Strategy

- **Kỹ thuật**: Biến đổi hình học qua **ImageDataGenerator**.
- **Các phép thử**: Xoay (15 độ), Lật ngang (Flip), Thu phóng (Zoom 10%), Dịch chuyển (Shift 10%).
- **Mục tiêu**: Giúp mô hình bất biến (Invariance) với sự thay đổi vị trí và góc nhìn của vật thể.
- **Nhãn**: Giữ nguyên nhãn gốc (Hard-label).

DATA PREPROCESSING

Tăng cường dữ liệu – CutMix Strategy

- **Kỹ thuật**: Cắt một vùng ảnh ngẫu nhiên từ lớp này dán đè lên lớp khác.
- **Cơ chế:** Tạo ảnh phức hợp + Trộn nhãn theo tỷ lệ diện tích (Soft-label).
- **Mục tiêu**: Chống Overfitting trên tập dữ liệu nhỏ (CIFAR-10)
- **Hiệu quả:** Kết hợp tốt nhất với **ResNet50 Fine-tuning** để tăng khả năng tổng quát hóa.



TRAINING & EVALUATION

8 tổ hợp thử nghiệm sẽ được thực hiện dựa trên các yếu tố:

- **Kiến trúc mô hình:** ResNet50 và Simple Vision Transformer (ViT).
- **Chiến lược huấn luyện:**
  - Fine-tune (huấn luyện toàn bộ)
  - Freeze Backbone (đóng băng bộ trích xuất đặc trưng).
- **Dữ liệu huấn luyện:**
  - Standard Augmentation
  - CutMix Augmentation.

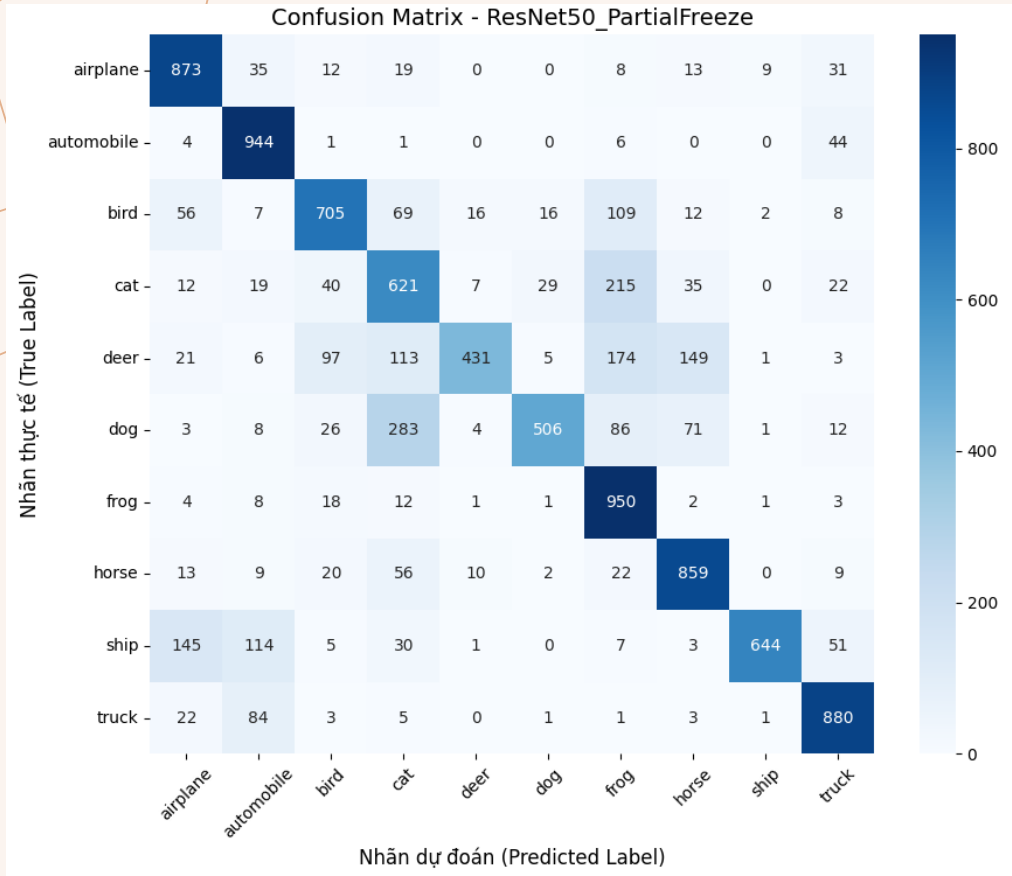
TRAINING & EVALUATION

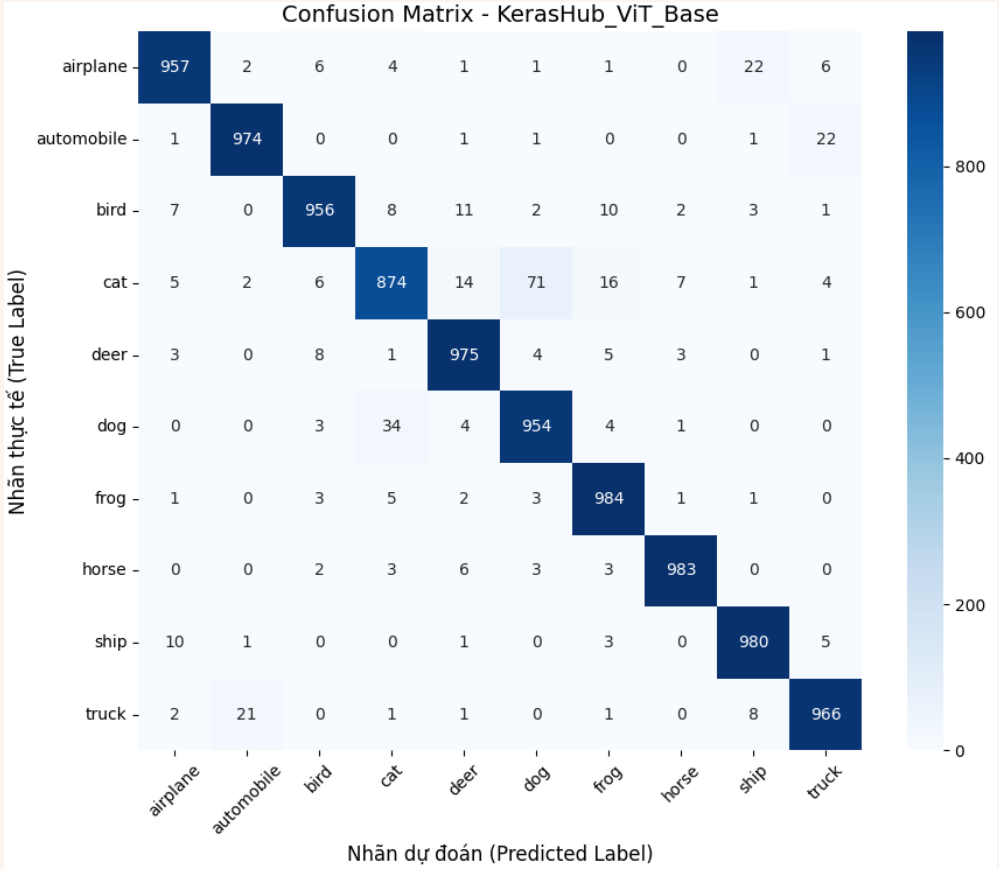
- Accuracy, Precision, Recall, F1-score

```
--- TIẾN HÀNH ĐÁNH GIÁ CHI TIẾT MÔ HÌNH: ResNet50_PartialFreeze ---
313/313 ━━━━━━━━━━━━━━━━━━━━ 37s 104ms/step

1. BÁO CÁO PHÂN LOẠI (CLASSIFICATION REPORT):
              precision    recall  f1-score   support

    airplane       0.76      0.87      0.81      1000
  automobile       0.76      0.94      0.85      1000
        bird       0.76      0.70      0.73      1000
         cat       0.51      0.62      0.56      1000
        deer       0.92      0.43      0.59      1000
         dog       0.90      0.51      0.65      1000
        frog       0.60      0.95      0.74      1000
       horse       0.75      0.86      0.80      1000
        ship       0.98      0.64      0.78      1000
       truck       0.83      0.88      0.85      1000

    accuracy                           0.74     10000
   macro avg       0.78      0.74      0.74     10000
weighted avg       0.78      0.74      0.74     10000
```

```
--- TIẾN HÀNH ĐÁNH GIÁ CHI TIẾT MÔ HÌNH: KerasHub_ViT_Base ---
313/313 ━━━━━━━━━━━━━━━━━━━━ 143s 445ms/step

1. BÁO CÁO PHÂN LOẠI (CLASSIFICATION REPORT):
              precision    recall  f1-score   support

    airplane       0.97      0.96      0.96      1000
  automobile       0.97      0.97      0.97      1000
        bird       0.97      0.96      0.96      1000
         cat       0.94      0.87      0.91      1000
        deer       0.96      0.97      0.97      1000
         dog       0.92      0.95      0.94      1000
        frog       0.96      0.98      0.97      1000
       horse       0.99      0.98      0.98      1000
        ship       0.96      0.98      0.97      1000
       truck       0.96      0.97      0.96      1000

    accuracy                           0.96     10000
   macro avg       0.96      0.96      0.96     10000
weighted avg       0.96      0.96      0.96     10000
```

TRAINING & EVALUATION





Confusion Matrix

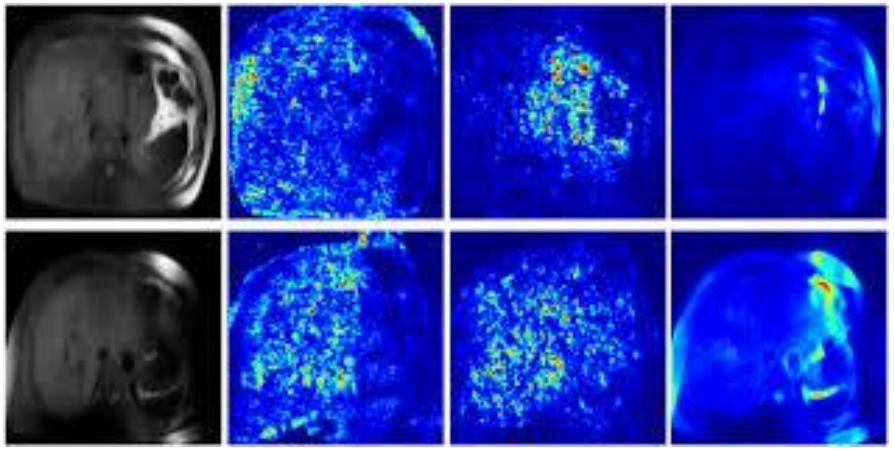
TRAINING & EVALUATION

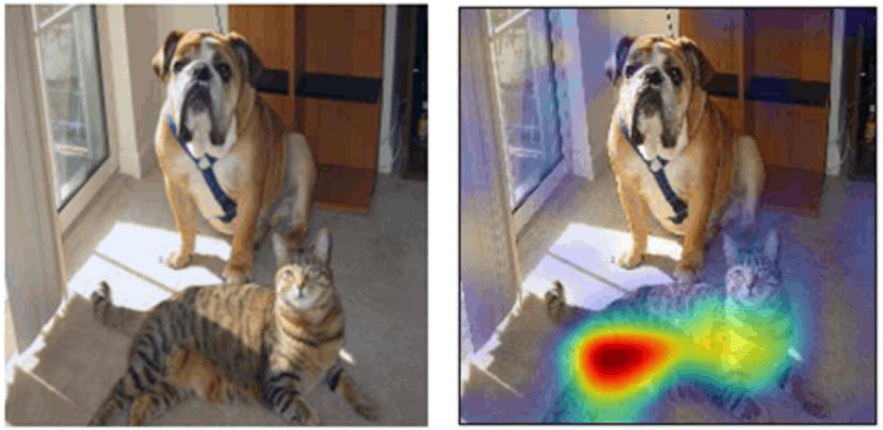
Nhận xét:

- **ViT – Global Attention:** Xử lý tốt ảnh phóng đại (Upsampling 224x224) nhờ khả năng nắm bắt cấu trúc tổng thể, ít bị ảnh hưởng bởi nhiễu pixel cục bộ.
- **ResNet50 – Local Filters**: Nhạy cảm với nhiễu và mờ cạnh khi ảnh bị kéo giãn 7 lần, khiến các bộ lọc 3x3 trích xuất đặc trưng sai lệch.

INTERPRETABILITY

- **CNN (ResNet50) - Grad-CAM:** Hiển thị Heatmap các vùng không gian quan trọng dựa trên luồng Gradient.
- **ViT - Vanilla Saliency Map:** Tính đạo hàm trực tiếp trên Pixel để xác định các điểm ảnh đóng góp lớn nhất vào dự đoán.

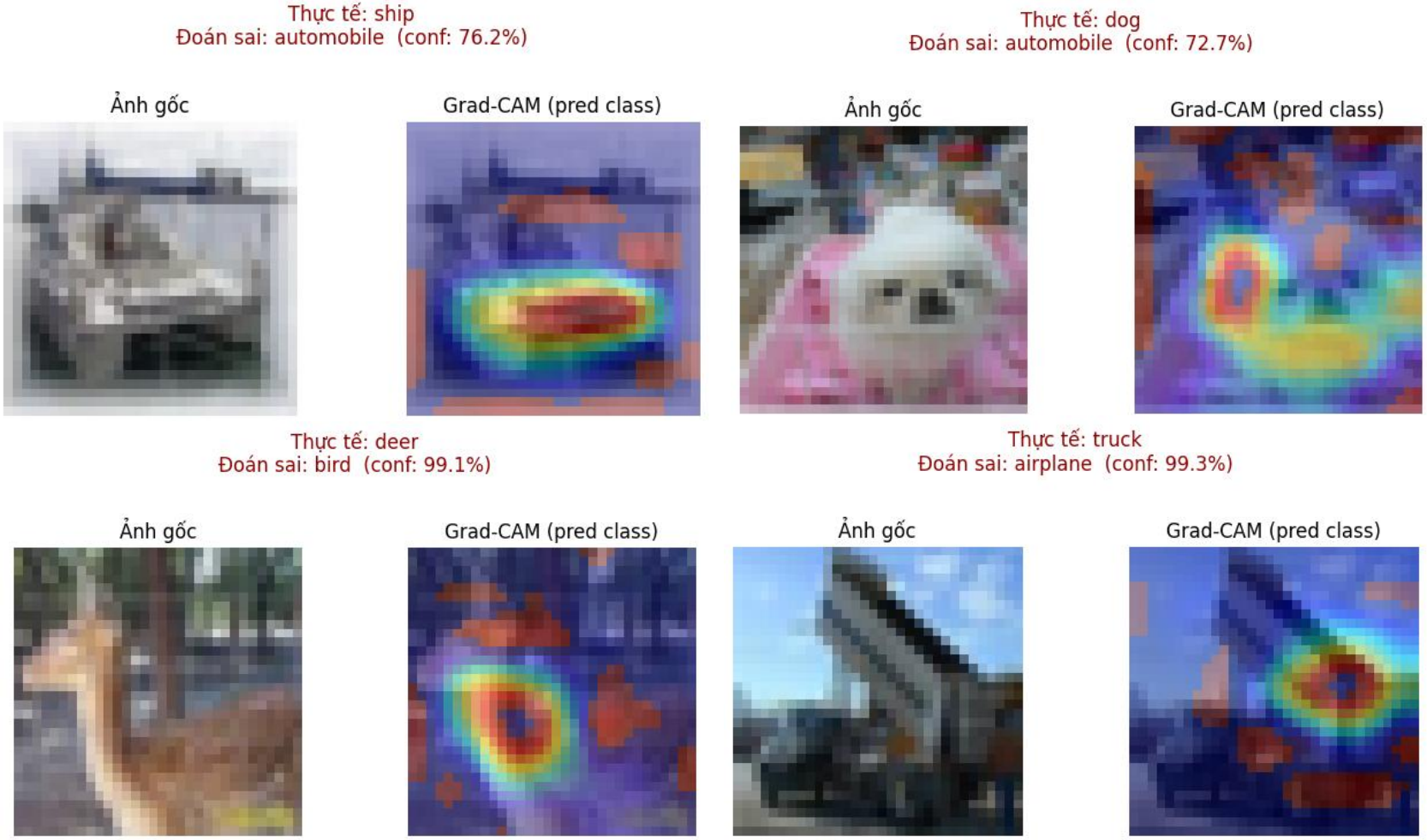




(a) Original Image (c) Grad-CAM 'Cat'

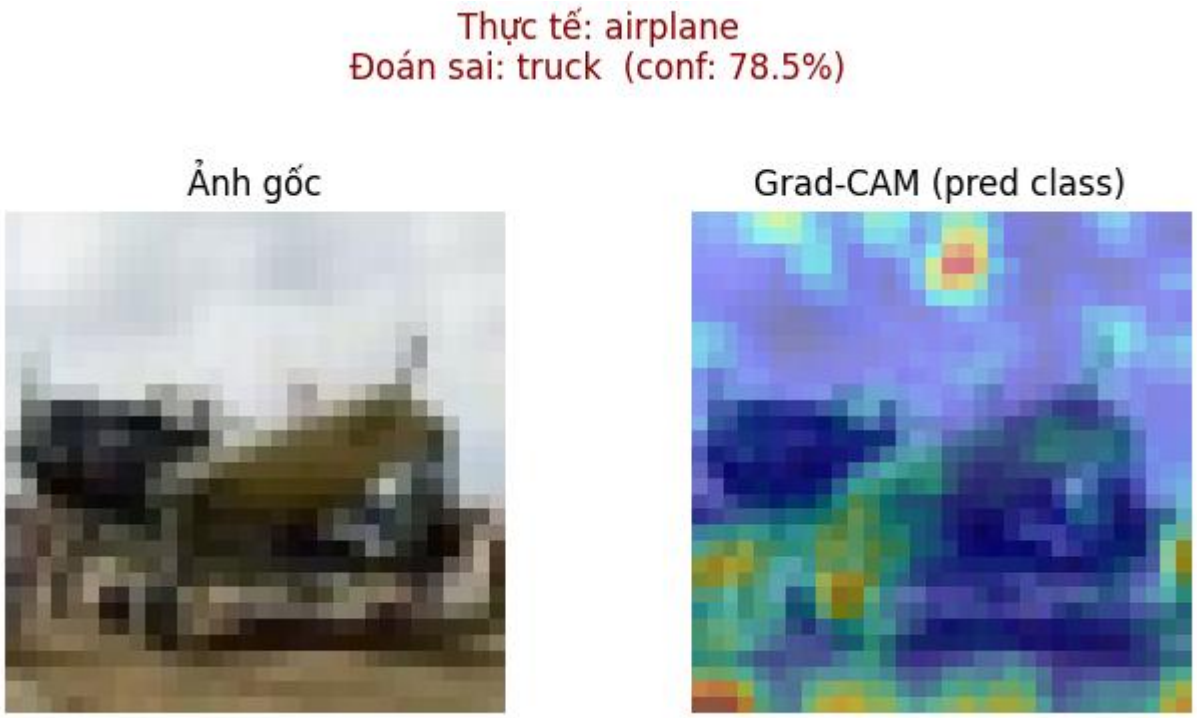
(g) Original Image (i) Grad-CAM 'Dog'

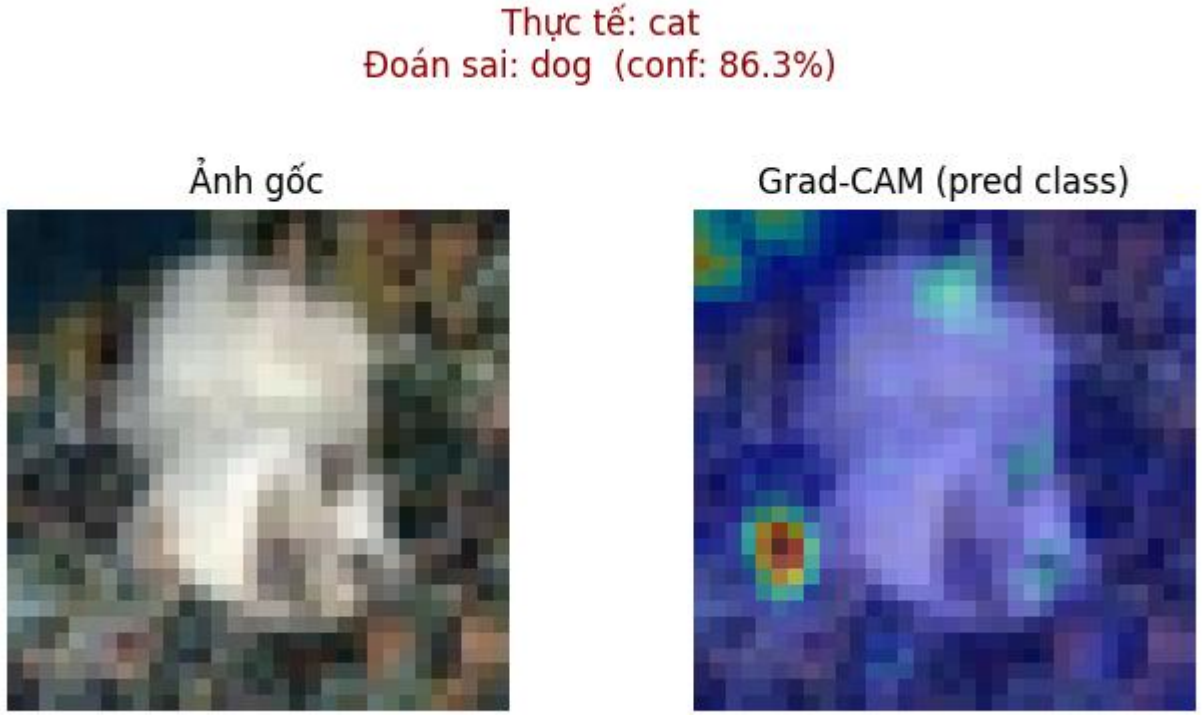
INTERPRETABILITY



ResNet50 with Grad-CAM

INTERPRETABILITY





ViT with Saliency Map

TRAINING & EVALUATION

Nhận xét:

- **ResNet50 (CNN):** Tập trung vào chi tiết cục bộ (cạnh, vân bề mặt) nhưng dễ bị nhiễu khi ảnh bị mờ do phóng đại.
- **ViT (Transformer):** Nắm bắt hình dáng tổng thể (Global shape) ổn định hơn nhờ cơ chế Self-Attention.

CONCLUSION

- Huấn luyện thành công ResNet50 và ViT trên CIFAR-10. Mô hình tối ưu nhất là **ResNet50 Fine-tune + CutMix**.
- **Data Augmentation** đóng vai trò quan trọng hơn cả việc chọn kiến trúc mô hình.
- ViT cần cơ chế Regularization mạnh và Learning Rate rất nhỏ để hội tụ.
- **Hạn chế:** Thời gian huấn luyện ViT còn dài, chưa thử nghiệm các kiến trúc lai (Swin Transformer) hoặc kỹ thuật Ensemble.

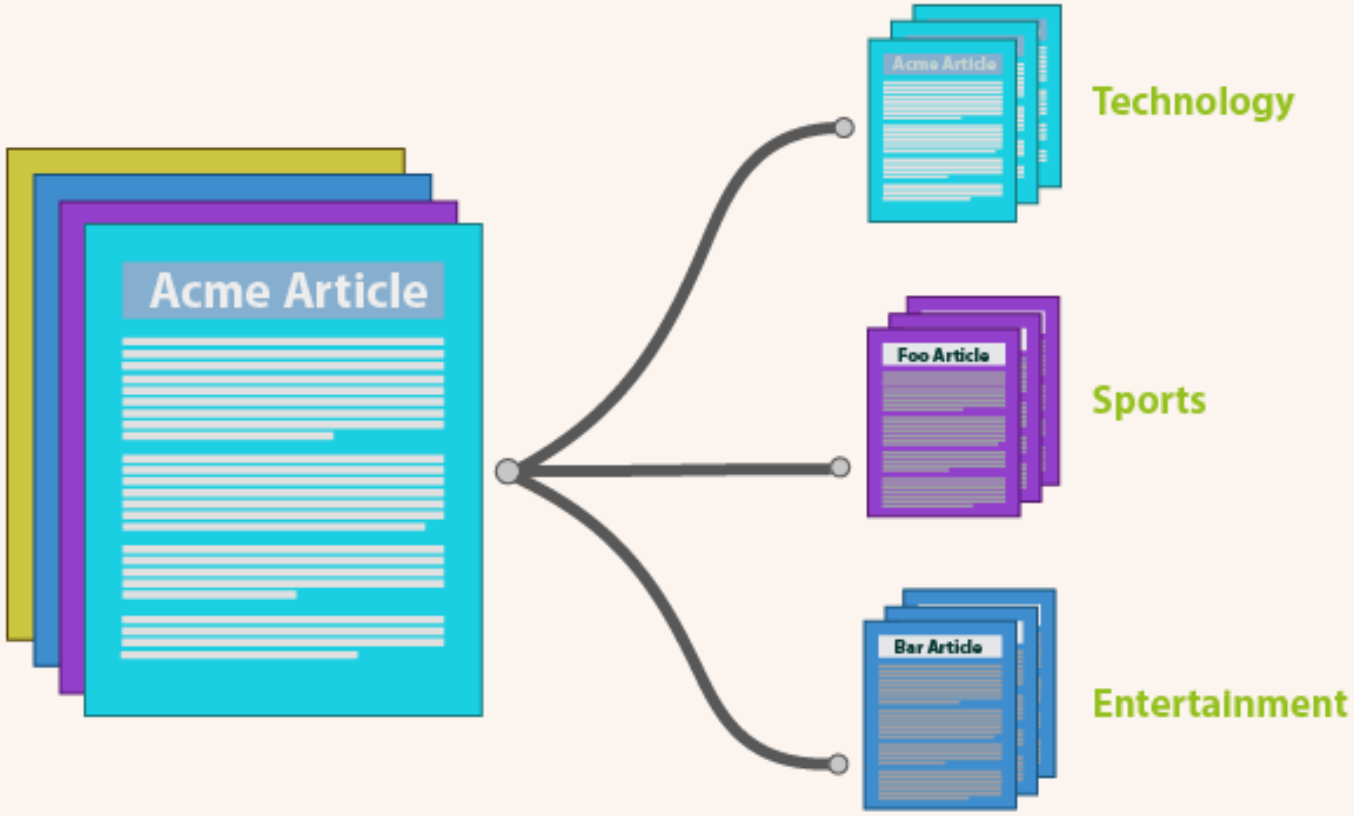
TEXT CLASSFICATION

INTRODUCTION

Text Classification là bài toán gán nhãn tự động cho một đoạn nội dung dựa trên ý nghĩa của nó.

Cơ chế: **Input:** Văn bản thô (Câu, đoạn văn, bài báo). **Output:** Nhãn/Lớp tương ứng (Category/Label).

INTRODUCTION

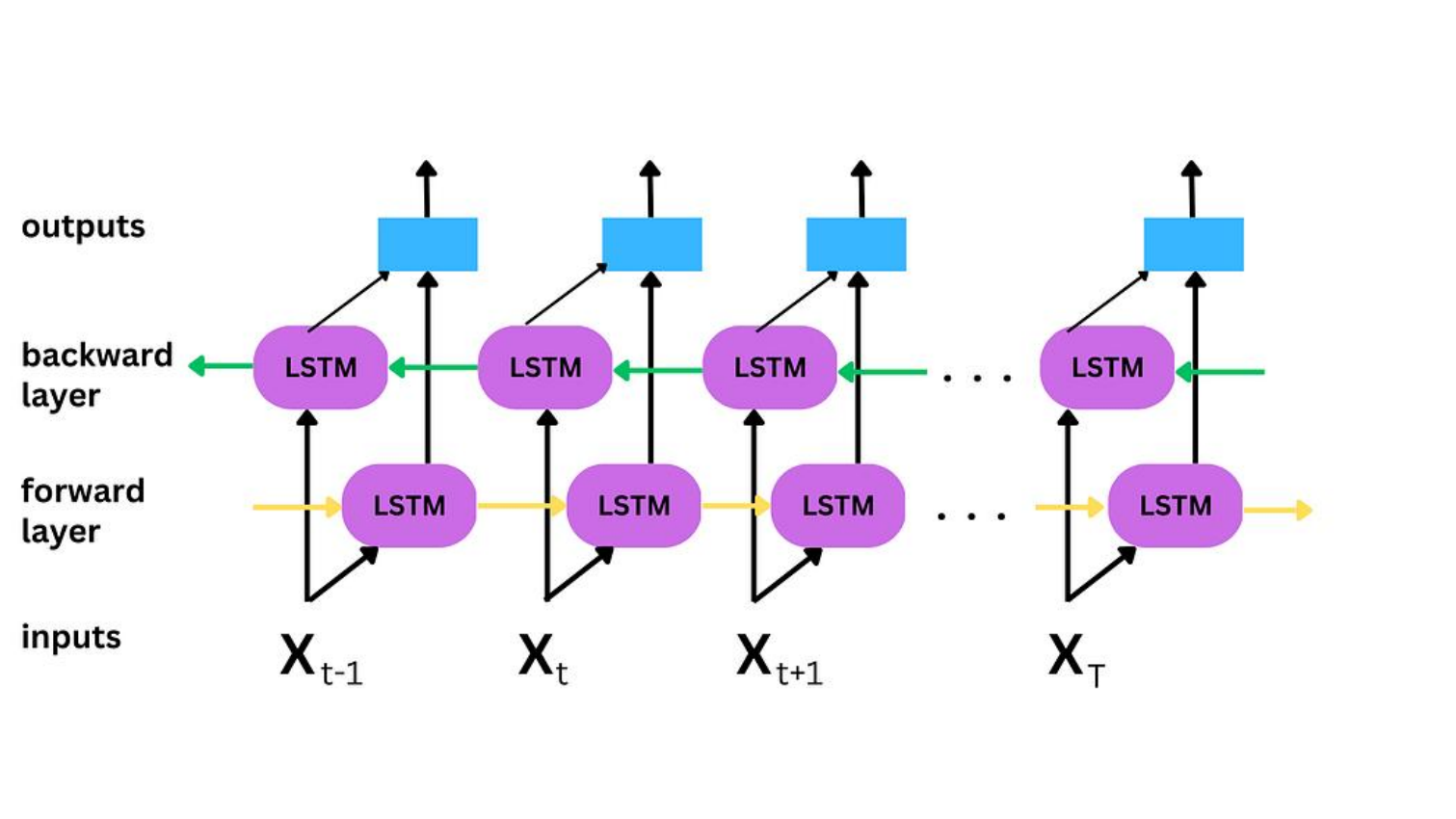


INTRODUCTION

Lựa chọn so sánh cho 2 loại kiến trúc:

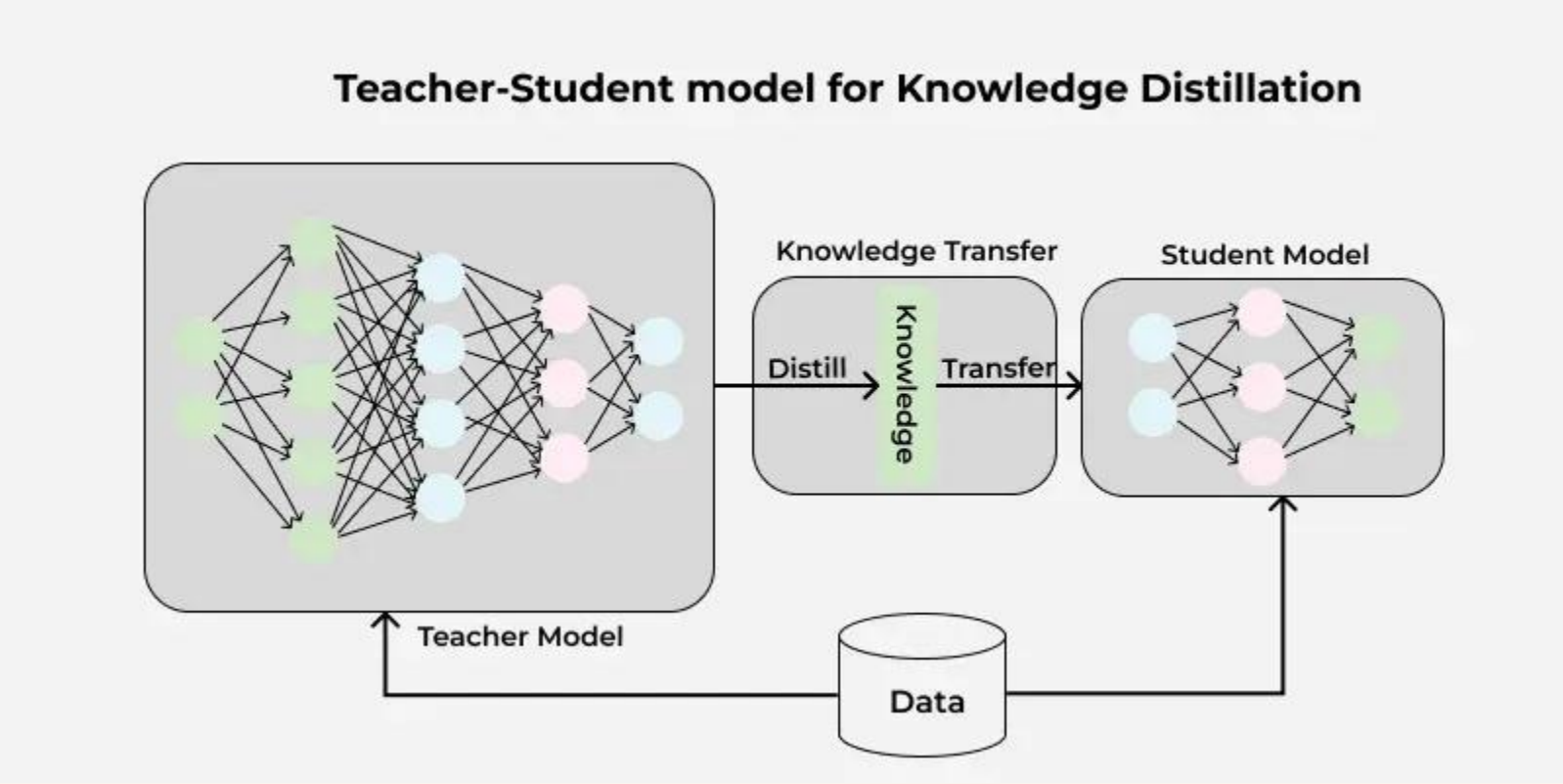
- **RNN (BiLSTM):** cải tiến của RNN chuyên dùng để xử lý dữ liệu dạng chuỗi (văn bản, âm thanh) nhờ khả năng ghi nhớ thông tin dài hạn và hiểu ngữ cảnh hai chiều.
- **Transformer Encoder (DistilBERT):** dựa trên kiến trúc Transformer, được thiết kế để giữ lại tối đa hiệu năng.

INTRODUCTION



RNN BiLSTM

INTRODUCTION



Transformer DistilBERT

DATASET

Yahoo! Answers Topic:

- **Nguồn:** https://www.kaggle.com/datasets/bhavikardeshna/yahoo-email-classification
- Bài toán phân loại văn bản nhiều lớp.
- **Quy mô:** 1.400.000 mẫu huấn luyện và 60.000 mẫu kiểm thử.
- **Cấu trúc:** Mỗi mẫu bao gồm 3 phần văn bản: Tiêu đề (Title), Nội dung câu hỏi (Content) và Câu trả lời hay nhất (Best Answer).
- **Số lớp:** 10 lớp chủ đề từ Xã hội, Khoa học đến Chính trị.

DATASET

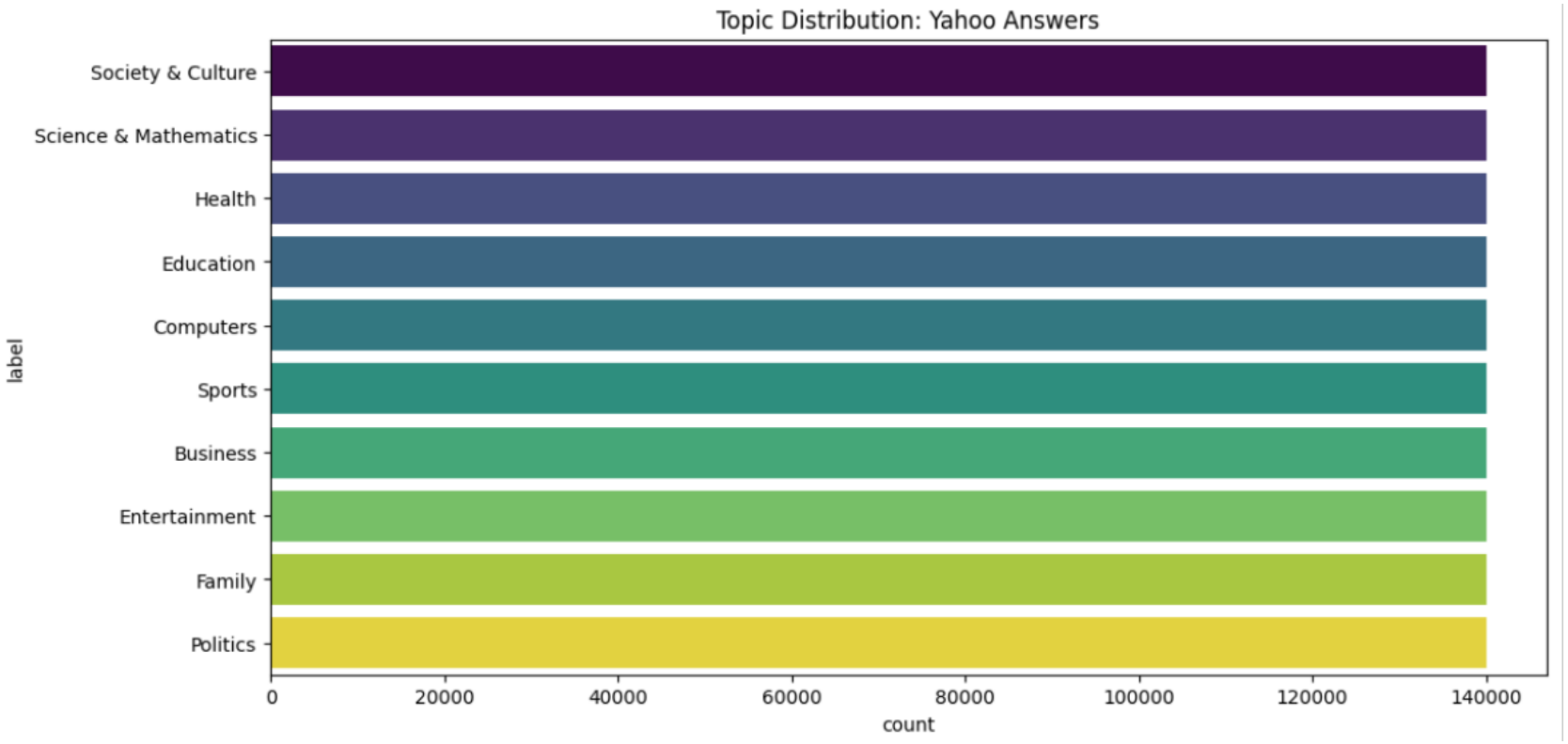
Vì sao chọn Yahoo! Answers Topic?

- Quy mô lớn & Cân bằng
- Văn bản đời thực chứa nhiều từ lóng, lỗi chính tả, viết tắt.
- 10 chủ đề có tập từ vựng giao thoa (ví dụ: Politics và Business), yêu cầu mô hình học được ngữ nghĩa sâu thay vì chỉ đếm từ khóa.

Lưu ý về dữ liệu

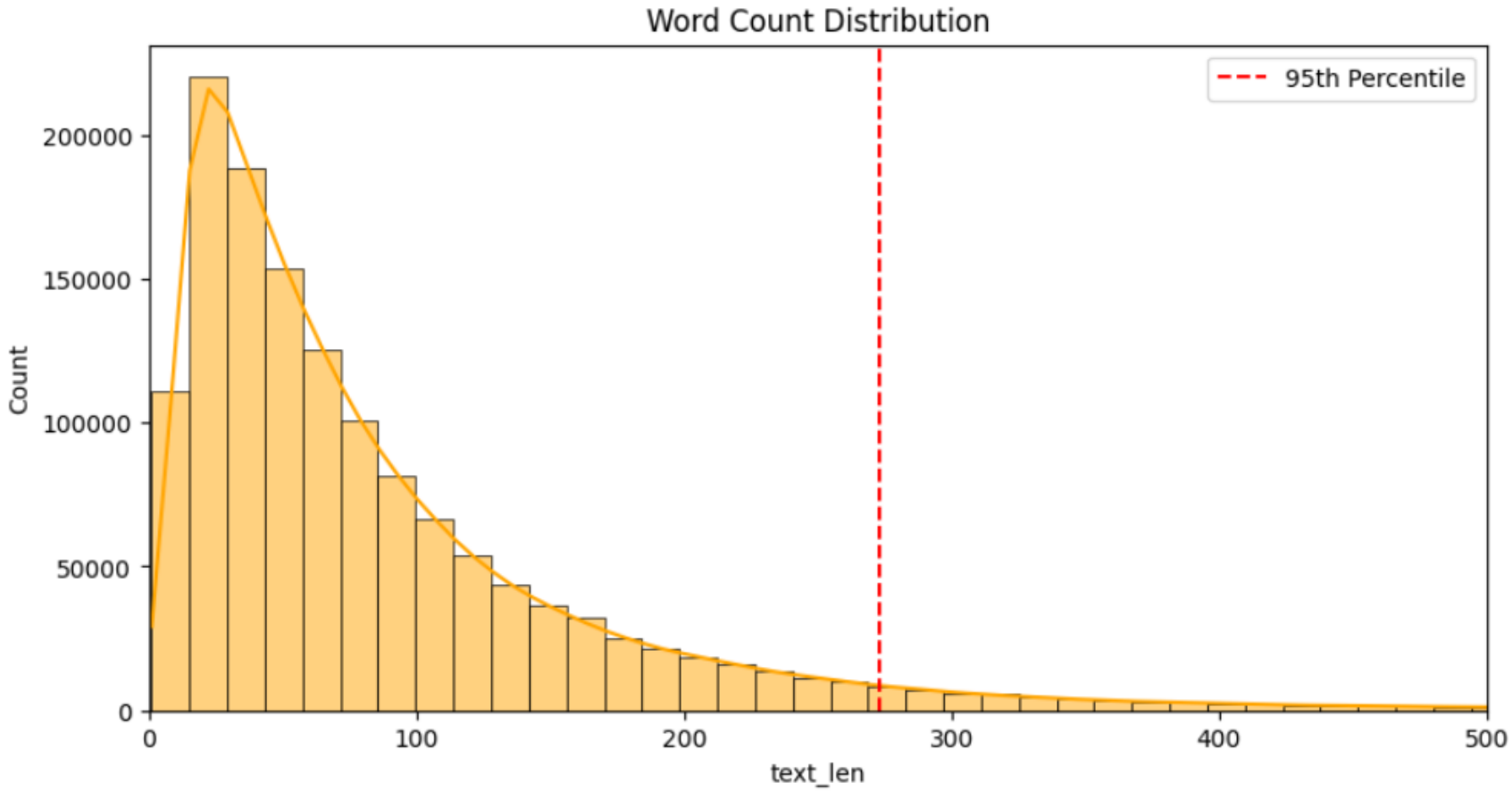
- Cần lọc tag HTML, ký tự đặc biệt và chuẩn hóa chữ viết tắt do đặc thù dữ liệu Internet.

EDA



Topic Distribution

EDA



Word Count Distribution

EDA

Sample question title by class

- **Society & Culture:** How does the Pope get his name?
- **Science & Mathematics:** Why do people blush when they are embarrassed?
- **Health:** What is Trans Fat? How to reduce that?
- **Education:** Who said the statement below and what does it mean?
- **Computers:** why doesn't an optical mouse work on a glass table?
- **Sports:** What is the best off-road motorcycle trail ?

EDA

Sample question title by class

- **Business:** How many planes Fedex has?
- **Entertainment:** Is Lin Qingxia (aka Brigitte Lin) "the most beautiful woman in Chinese cinema?
- **Family:** what is the reason for the increasing divorce percentage in the western world?
- **Politics:** Is there a good reference for California Landlord-Tenant Law?

Note: Dữ liệu huấn luyện thực tế bao gồm Title + Content + Best Answer

DATA PREPROCESSING

- **Gộp nội dung (Concatenation):** Kết hợp Title + Content + Answer thành 1 chuỗi để mô hình học được ngữ cảnh đầy đủ nhất.
- **Làm sạch văn bản (Cleaning):** Chuyển về chữ thường và dùng Regex loại bỏ các thẻ HTML rác (
, ") giúp đồng nhất dữ liệu.
- **Loại bỏ nhiễu:** Xóa các mẫu trùng lặp và dòng trống sau khi làm sạch để tránh mô hình bị "học vẹt".
- **Phân tách cân bằng (Stratified Split):** Chia tập Train/Validation theo tỷ lệ 9:1, đảm bảo sự cân bằng tuyệt đối giữa 10 lớp chủ đề.

RNN – BILSTM

Ý tưởng:

- Sơ đồ kiến trúc: Nhúng Tokens (Embedding 128D) → Bi-LSTM (Học 2 chiều) → Ghép Hidden States (256D) → Bộ lọc (Dropout) → Phân loại
- Xử lý tuần tự từng token, kết hợp bộ nhớ dài hạn (Cell State) và ghép ngữ cảnh hai chiều để nắm bắt thông tin toàn cục của văn bản

RNN – BILSTM

Pipeline:

- Embedding Layer: input_dim=vocab_size, output_dim=128
- BiLSTM: 2 chiều, hidden_units=128, concatenate=256
- Dropout: 0.2, tăng khả năng tổng quát hóa
- Linear Classifier: Input (256) → Output (10)
- Training: AdamW optimizer (lr=1e-3), CrossEntropyLoss, Early Stopping (Patience: 5, Restore: Best Model Weights)

TRANSFORMER – DISTILBERT

Ý tưởng:

- Tính toán song song (Self-Attention), nhìn vào toàn bộ câu cùng lúc để xác định mối quan hệ giữa các từ.
- Sơ đồ kiến trúc: tất cả tokens xử lý đồng thời → DistilBERT (Pre-trained) → Vector tóm tắt (768D) → Bộ lọc (Dropout) → Phân loại

TRANSFORMER - DISTILBERT

Pipeline:

- Chuyển văn bản thành ID + Attention Mask (độ dài 150 tokens).
- Pre-trained DistilBERT: Sử dụng mô hình đã học sẵn (12 layers, 768 chiều)
- Freeze Backbone: Giúp giữ nguyên tri thức ngôn ngữ từ Google.
- CLS-Pooling: Lấy Vector đại diện của token đầu tiên ([CLS]) làm tóm tắt cho toàn bộ ý nghĩa câu hỏi.
- Dropout: 0.1 (Mức mặc định tối ưu của Transformer để chống học vẹt).
- Linear Classifier: Input (768) → Output (10)
- Training: AdamW (lr=2e-5), CrossEntropyLoss, Early Stopping (Patience: 2)

TRAINING & EVALUATION

- Accuracy, Precision, Recall, F1-score, Inference Latency

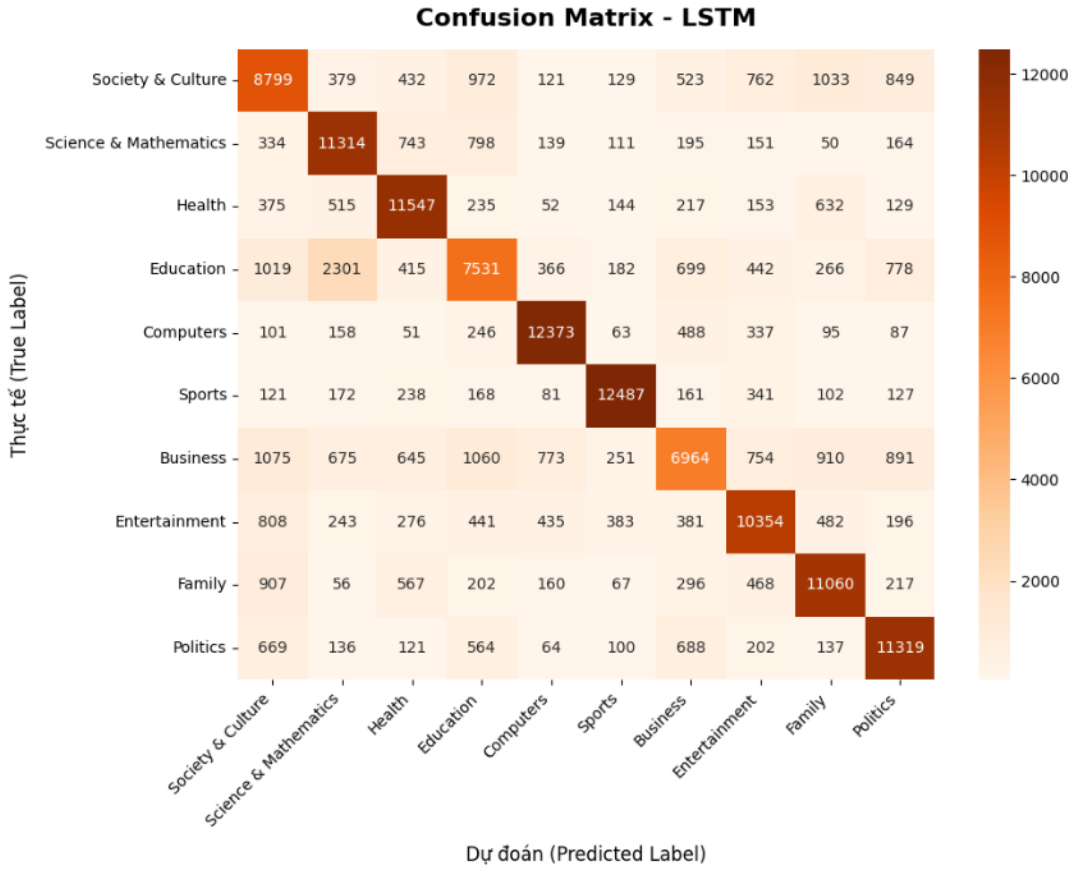
Chỉ số đánh giá	Bi-LSTM (Full Training)	DistilBERT (Freeze)	Chênh lệch
Accuracy	**0.7411**	0.7111	+3.00%
Precision (Macro)	**0.7374**	0.7060	+3.14%
Recall (Macro)	**0.7411**	0.7111	+3.00%
F1-Score (Macro)	**0.7373**	0.7060	+3.13%
Inference Latency	**0.3064 ms**	4.8758 ms	**Nhanh x15.9**

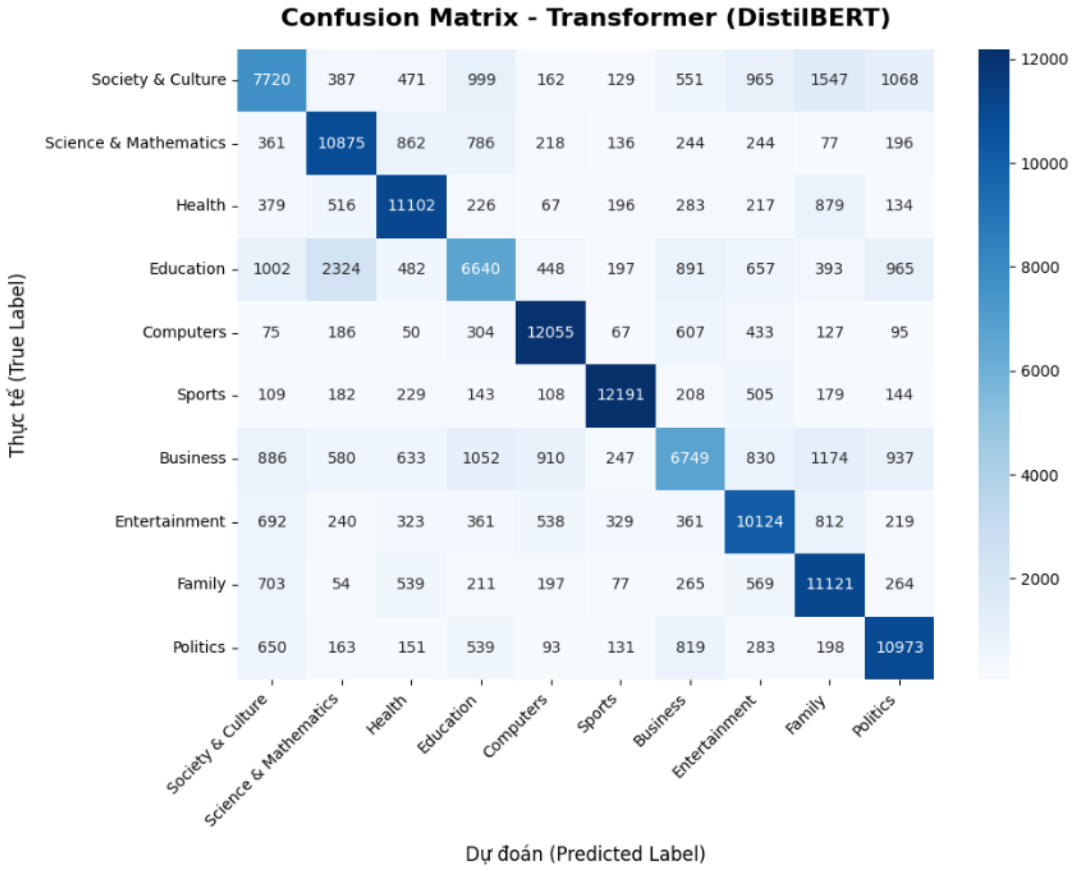
TRAINING & EVALUATION

Nhận xét:

- **Bi-LSTM (74.11%):** Hiệu năng cao, hội tụ nhanh và tương thích tốt khi huấn luyện từ đầu.
- **Hạn chế của DistilBERT (71.11%):** Thấp hơn do áp dụng cơ chế Freeze Backbone, chỉ được tinh chỉnh ở tầng phân loại cuối.
- **Bi-LSTM** thể hiện ưu thế về tốc độ xử lý và yêu cầu phần cứng thấp nhưng vẫn đảm bảo độ chính xác ổn định.
- Kết quả của DistilBERT có thể cải thiện nếu thực hiện Unfreeze và tăng số Epoch

ERROR ANALYSIS





Confusion Matrix

ERROR ANALYSIS

Top cặp nhầm lẫn:

RNN - BiLSTM

Education → Science & Mathematics (2301)

Business → Society & Culture (1076)

Business → Education (1060)

Society & Culture → Family (1031)

Education → Society & Culture (1020)

Transformer - DistilBERT

Education → Science & Mathematics (2324)

Society & Culture → Family (1547)

Business → Family (1174)

Society & Culture → Politics (1068)

Business → Education (1052)

Nhận xét:

- Cặp Education & Science nhầm lẫn nặng nhất (>2300 mẫu) do dùng chung thuật ngữ học thuật.
- Các chủ đề như Business, Education và Society thường xuyên nhầm lẫn chéo do ranh giới ngữ nghĩa mờ nhạt giữa các câu hỏi thảo luận chung.

INTERPRETABILITY

Phương pháp:

- Sử dụng Gradient-based Attribution (tính đạo hàm)
- Tính toán gradient của Output dựa theo lớp Token Embedding.
- Token có gradient lớn → ảnh hưởng cao hơn

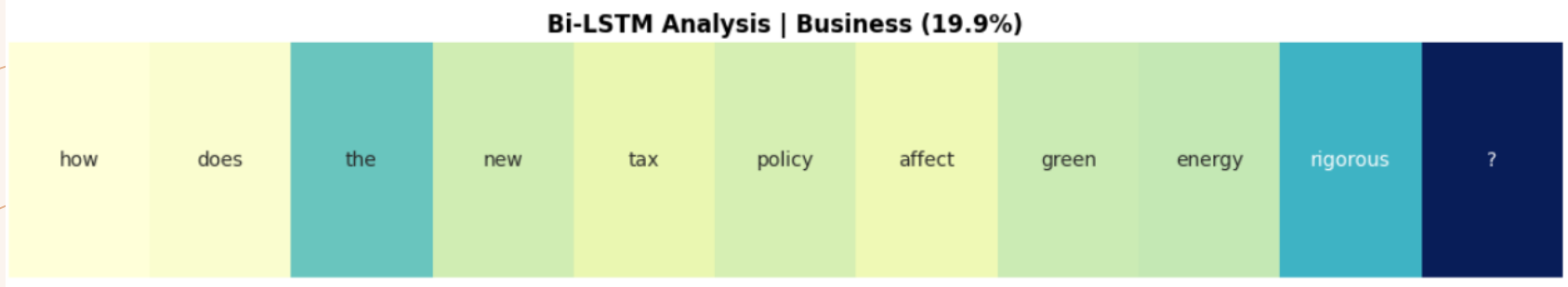
Ý nghĩa:

- Token hợp lý → Mô hình đang học đúng đặc trưng ngữ nghĩa.
- Token lạ/nhiễu → Mô hình đang bị "học vẹt" hoặc phụ thuộc vào nhiễu dữ liệu.
- Giúp debug và hiểu cách mô hình suy luận

INTERPRETABILITY

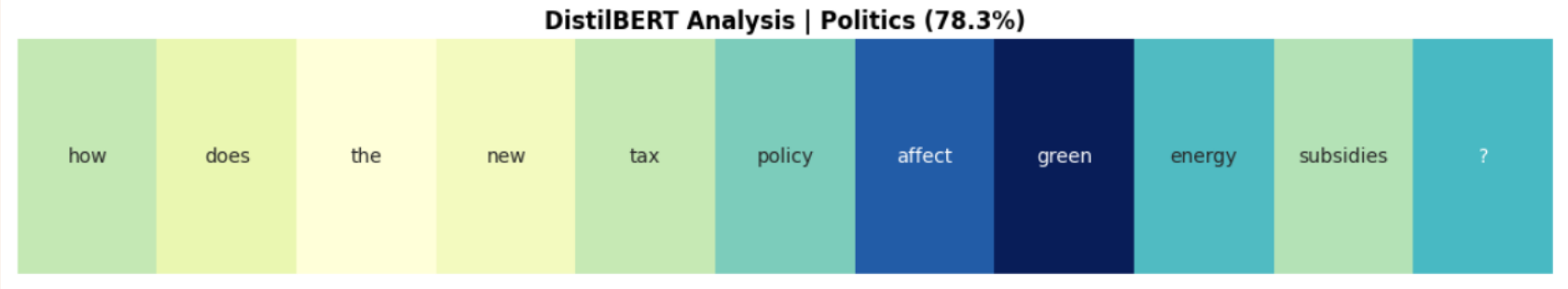
Sample Query: "How does the new tax policy affect green energy subsidies?"

Dự đoán của mô hình RNN - BiLSTM



1. Business: **19.85%**
2. Entertainment: 13.01%
3. Family: 12.45%

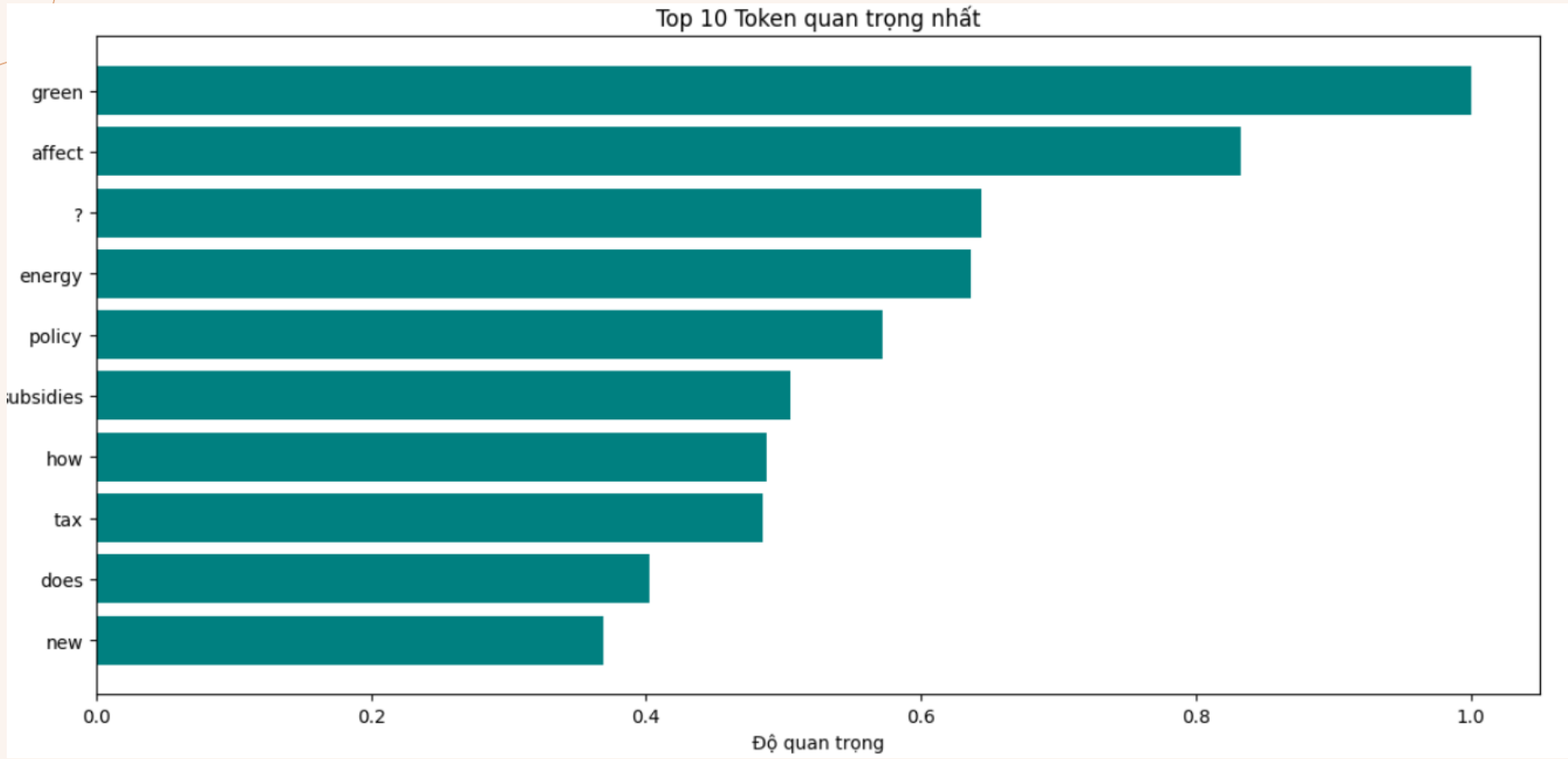
Dự đoán của mô hình Transformer - DistilBERT



1. Politics: **78.32%**
2. Education: 9.35%
3. Business: 9.22%

INTERPRETABILITY

Sample Query: “How does the new tax policy affect green energy subsidies?”



TUNING STRATEGY

- **Freeze Backbone:** Đóng băng toàn bộ thân DistilBERT, chỉ huấn luyện tầng phân loại cuối để tiết kiệm tài nguyên và giữ nguyên tri thức gốc.
- **Full Fine-tuning:** Mở khóa tất cả các tầng để cập nhật toàn bộ tham số, giúp mô hình thích nghi sâu và đạt độ chính xác tối đa trên dữ liệu mới.
- **Layer-wise LR:** Sử dụng tốc độ học (LR) khác nhau cho từng phần để cân bằng giữa việc giữ kiến thức cũ và học nhanh kiến thức mới.

TUNING STRATEGY

Nhận xét:

- LAYER_WISE (tốc độ học đa tầng) đạt F1-Score cao nhất (0.7632) nhờ tinh chỉnh sâu vào các lớp Transformer giúp mô hình đạt độ chính xác cao hơn ~13% so với phương pháp FREEZE.
- Có sự đánh đổi rõ rệt giữa thời gian và chất lượng. Hai kịch bản FULL và LAYER_WISE cần thời gian huấn luyện gấp đôi so với kịch bản FREEZE.

Chiến lược	Accuracy	F1-Score	Thời gian (giây)	Thời gian (phút)
FREEZE	0.6432	0.6354	5481.89	91.36
FULL	0.7634	0.7596	8648.18	144.14
LAYER_WISE	0.7669	0.7632	8748.26	145.80

CONCLUSION

- DistilBERT (Layer-wise LR) đạt kết quả tốt nhất (F1-score = 76.32%, vượt trội Bi-LSTM khoảng 15% nhờ cơ chế Self-Attention xử lý ngữ cảnh dài.
- DistilBERT có độ tự tin cao (>70%) và kháng nhiễu tốt, trong khi Bi-LSTM dễ bị phân tán xác suất (<20%) và nhầm lẫn các chủ đề phổ thông.
- Saliency Map xác nhận DistilBERT tập trung vào từ khóa mang tính ngữ nghĩa (logic), Bi-LSTM phụ thuộc vào tần suất token đơn lẻ.
- DistilBERT là lựa chọn tối ưu cho phân loại đa lớp.
- Bi-LSTM có thể tối ưu bằng cách tích hợp cơ chế Attention, kết hợp CNN (Hybrid) hoặc sử dụng các tầng nhúng chuyên sâu (ELMo/FastText).

MULTIMODAL CLASSFICATION

INTRODUCTION

- **Mục tiêu:** phân loại đa phương thức với đầu vào là cặp (image, caption). Multimodal giúp giảm nhập nhằng so với chỉ ảnh hoặc chỉ text
- So sánh 2 kiến trúc:
  - Zero-shot: không cần nhãn huấn luyện
  - Few-shot: chỉ cần ít nhãn (K mẫu/lớp) + linear probe

DATASET

Flickr30k

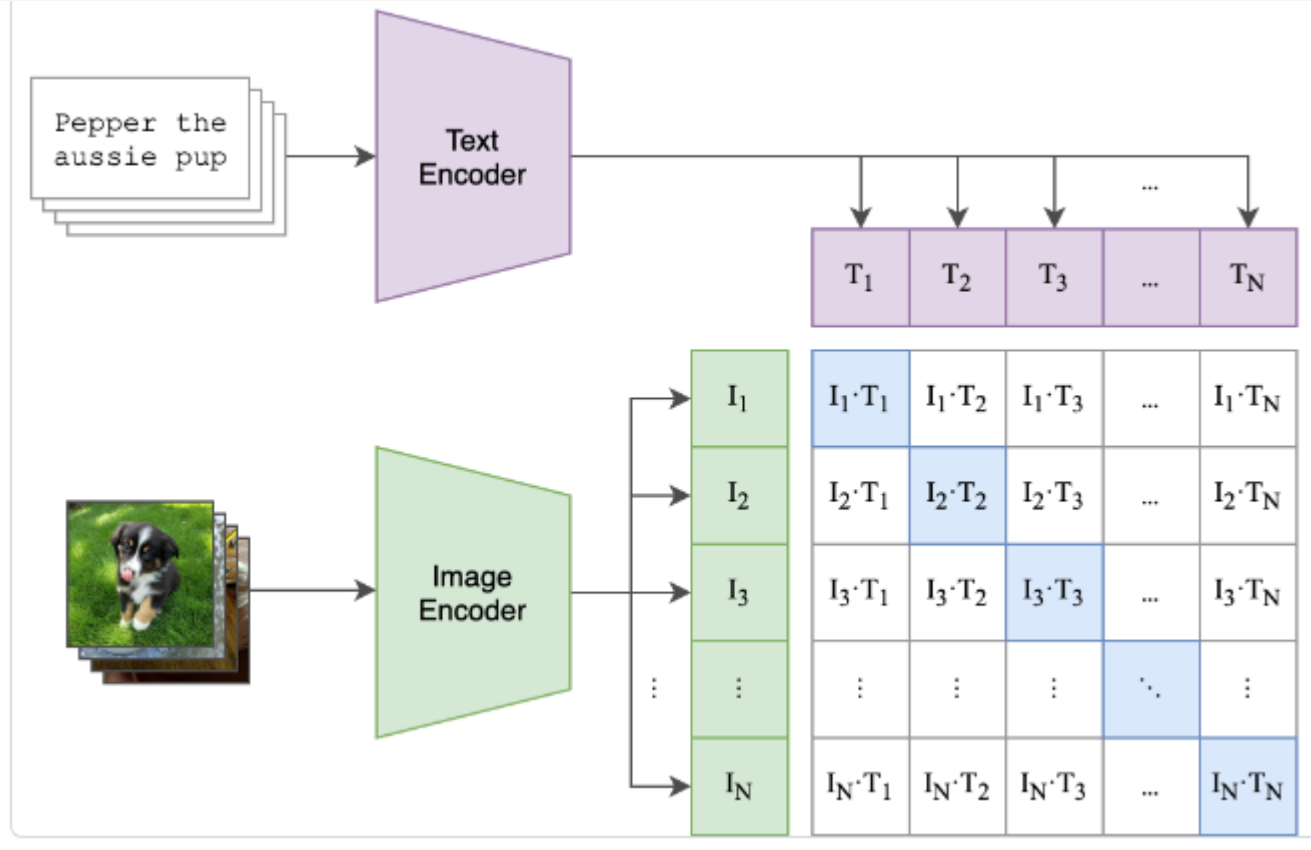
- **Nguồn:** https://www.kaggle.com/datasets/hsankesara/flickr-image-dataset
- Bộ dữ liệu tiêu chuẩn trong các bài toán Image Captioning và Multi-modal Retrieval (tìm kiếm đa phương thức).
- **Quy mô:** Bao gồm 31,783 hình ảnh được thu thập từ trang web Flickr.
- **Đặc điểm:** Mỗi hình ảnh không chỉ có một nhãn đơn lẻ, mà đi kèm với 5 câu mô tả (captions) khác nhau do con người viết.
- Kiểm tra khả năng hiểu ngữ cảnh phức tạp của mô hình (không chỉ là vật thể mà còn là hành động, màu sắc, mối quan hệ không gian)

CLIP

Mô hình CLIP (Contrast Language Image Pre-training)

- Được OpenAI giới thiệu vào năm 2021
- **Cấu trúc gồm hai phần (Two-tower architecture):**
  - **Image Encoder** (ResNet, ViT)
  - **Text Encoder** (Transformer)
- **Cơ chế:** CLIP sẽ cực đại hóa độ tương đồng (cosine similarity) của cặp đúng và cực tiểu hóa độ tương đồng của cặp sai.

CLIP


Pepper the aussie pup
Text Encoder
Image Encoder
T_1 T_2 T_3 ... T_N
I_1 I_2 I_3 ⋮ I_N
$I_1 \cdot T_1$ $I_1 \cdot T_2$ $I_1 \cdot T_3$ ... $I_1 \cdot T_N$
$I_2 \cdot T_1$ $I_2 \cdot T_2$ $I_2 \cdot T_3$ ... $I_2 \cdot T_N$
$I_3 \cdot T_1$ $I_3 \cdot T_2$ $I_3 \cdot T_3$ ... $I_3 \cdot T_N$
$I_N \cdot T_1$ $I_N \cdot T_2$ $I_N \cdot T_3$ ... $I_N \cdot T_N$

CLIP – ZERO SHOT CLASSIFICATION

Quy trình:

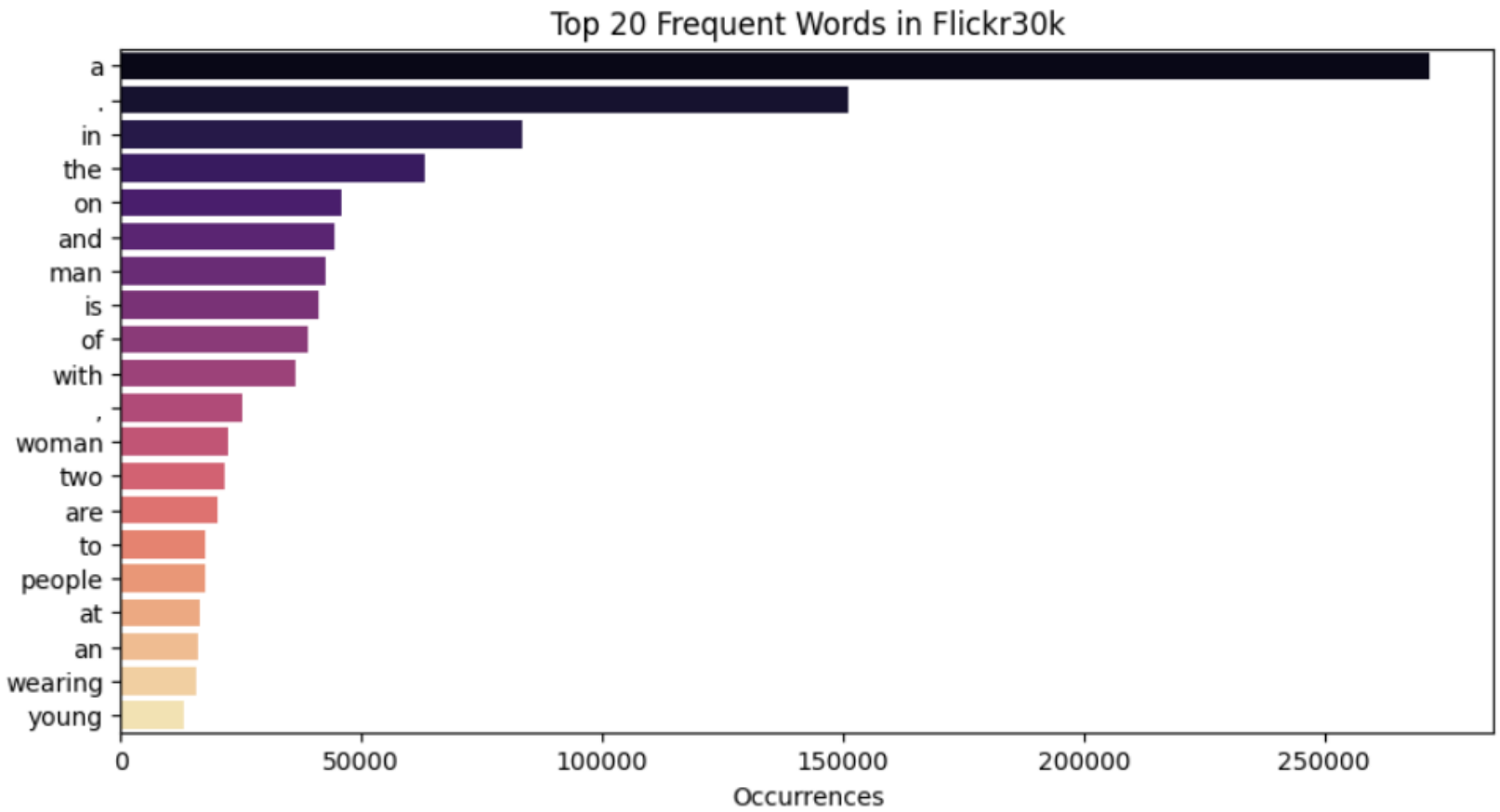
- **Tạo Prompt:** Thay vì dùng nhãn đơn lẻ như "dog", ta đưa vào một template prompt như: "*A photo of a [label]*".
- **Vector hóa nhãn:** Tất cả các labels tiềm năng sẽ được đưa qua Text Encoder để tạo ra các vector embedding văn bản.
- **So khớp:** Ảnh mới được đưa qua Image Encoder. CLIP sẽ tính toán vector ảnh này "gần" với vector của nhãn nào nhất trong không gian vector. Nhãn có độ tương đồng cao nhất sẽ là kết quả dự đoán.

Note: Zero-shot biến bài toán phân loại (Classification) thành bài toán tìm kiếm ngữ nghĩa (Semantic Search).

CLIP – FEW SHOT CLASSIFICATION

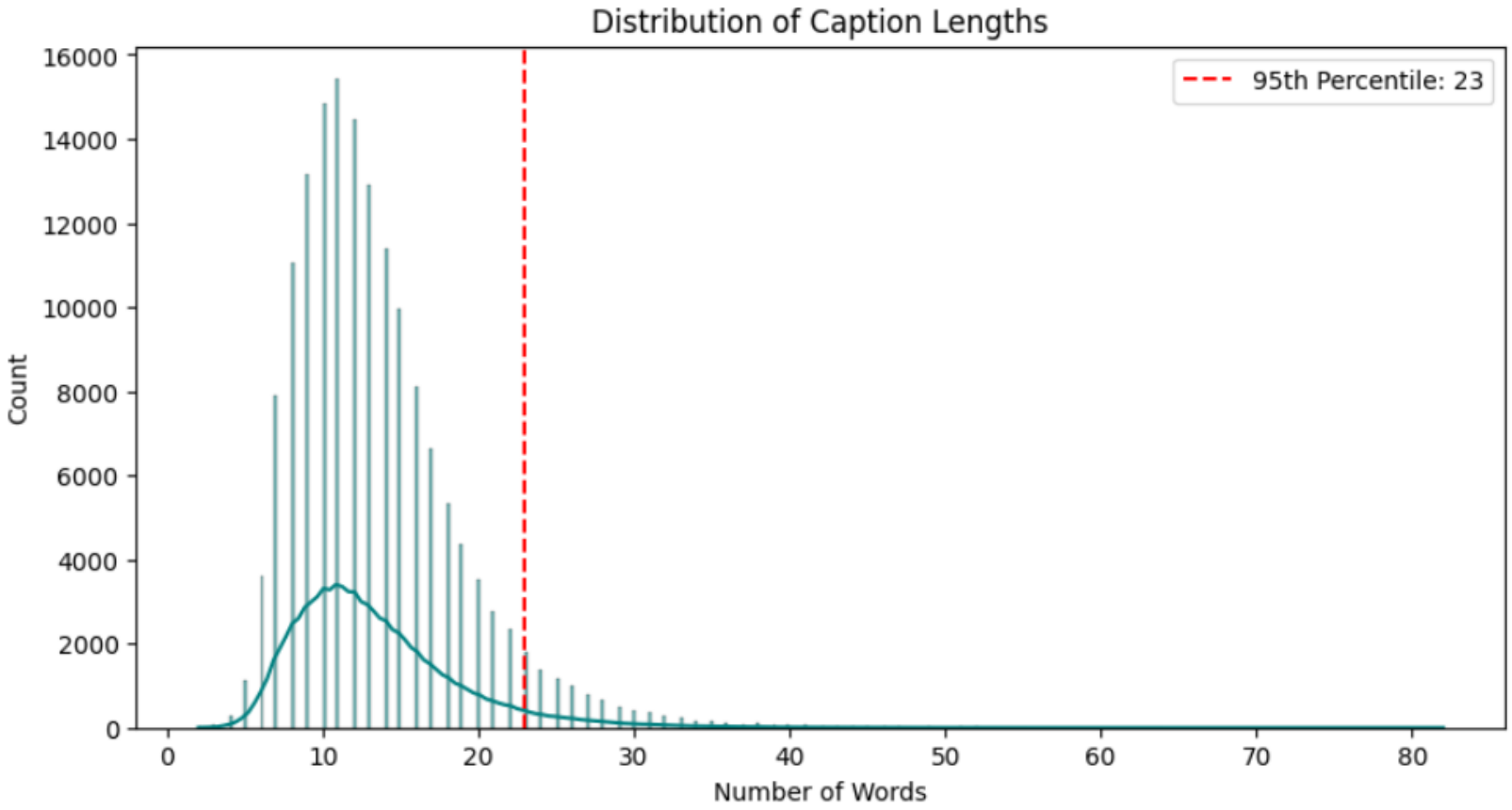
- Sử dụng phương pháp Linear Probing.
- **Cách hoạt động:**
  - **Đóng băng (Freeze):** Giữ nguyên toàn bộ trọng số của CLIP (Image và Text Encoder). CLIP như một bộ trích xuất đặc trưng (Feature Extractor).
  - **Trích xuất đặc trưng:** Đưa tập dữ liệu nhỏ của qua Image Encoder để thu vector đặc trưng.
  - **Huấn luyện bộ phân loại tuyến tính:** Sử dụng một thuật toán ML thường (**Logistic Regression)**, để học cách phân loại các vector đặc trưng này vào các nhãn mong muốn.

EDA



Word Frequency

EDA



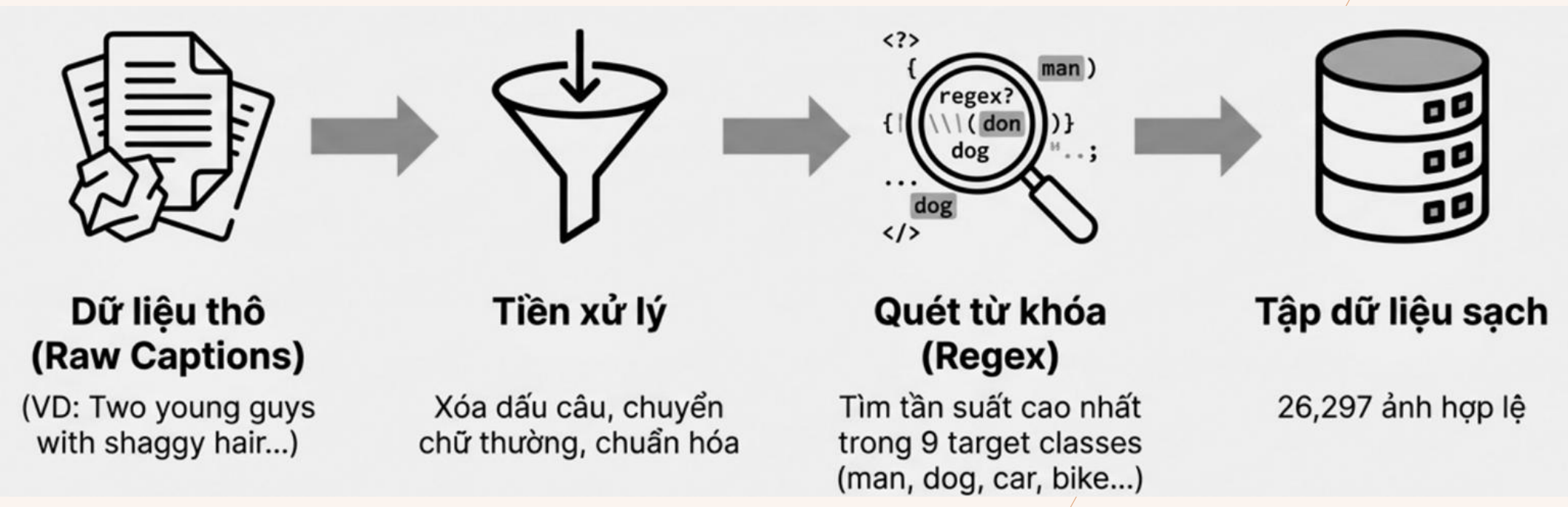
Caption Length Distribution

DATA PREPROCESSING

Gồm 4 giai đoạn:

- Làm sạch văn bản (Text Cleaning)
- Lọc và Kiểm tra tính hợp lệ
- Gán nhãn tự động (Weak Labeling)
- Chuẩn bị dữ liệu cho Mô hình

DATA PREPROCESSING



DATA PREPROCESSING

Làm sạch văn bản:

- Chuyển về lowercase
- Loại bỏ ký tự đặc biệt/số bằng Regex để chuẩn hóa đầu vào cho Text Encoder

```python
def clean_text(text):
    # Convert to string and lowercase
    text = str(text).lower()

    # Remove punctuation and numbers using regex
    # Only keep alphabetic characters and spaces
    text = re.sub(r'[^a-z\s]', '', text)

    # Remove extra whitespaces
    text = " ".join(text.split())
    return text

# Apply the cleaning function to create 'clean_caption' column
df['clean_caption'] = df['comment'].apply(clean_text)
```

DATA PREPROCESSING

```
df_grouped = df.groupby('image_name')['clean_caption'].first().reset_index()
df_grouped.columns = ['image', 'caption']
```

Xử lý đa chú thích:

- Group dữ liệu theo image_name
- Lấy caption đầu tiên làm đại diện để đảm bảo tính nhất quán (31,783 ảnh hợp lệ)

```
# Lọc ảnh tồn tại
valid_rows = []
for idx, row in df_grouped.iterrows():
    img_path = os.path.join(image_path, row['image'])
    if os.path.exists(img_path):
        valid_rows.append({
            'path': img_path,
            'caption': row['caption'],
            'image_name': row['image']
        })


df_valid = pd.DataFrame(valid_rows)
print(f"Tổng số ảnh hợp lệ: {len(df_valid)}")
```

```
Tổng số ảnh hợp lệ: 31783
```

DATA PREPROCESSING

Định nghĩa nhãn mục tiêu:

- Thiết lập 9 lớp phổ biến *(man, dog, car, bike, woman, water, tree, building, ball)*

```python
# Định nghĩa nhãn và regex tương ứng
LABEL_MAP = {
    "man": r"\b(man|men|guy|guys|boy|boys|male)\b",
    "dog": r"\b(dog|dogs|puppy|puppies)\b",
    "car": r"\b(car|cars|vehicle|autos)\b",
    "bike": r"\b(bike|bikes|bicycle|bicycles|motorcycle)\b",
    "woman": r"\b(woman|women|girl|girls|lady|ladies|female)\b",
    "water": r"\b(water|ocean|sea|lake|river|pool|beach|surf|shore)\b",
    "tree": r"\b(tree|trees|forest|woods|grass|field|garden|nature|plant)\b",
    "building": r"\b(building|house|skyscraper|street|sidewalk|city|structure|wall)\b",
    "ball": r"\b(ball|basketball|soccer|football|baseball|tennis|volleyball)\b",
}

TARGET_CLASSES = list(LABEL_MAP.keys())
print(TARGET_CLASSES)
```

DATA PREPROCESSING

Kỹ thuật Weak Labeling:

- Sử dụng Regex để tự động gán nhãn dựa trên từ khóa trong caption

```
Số lượng ảnh sau lọc: 26297
label
man          13042
woman         6190
dog           1806
building      1567
water         1096
ball           957
tree           699
bike           574
car            366
Name: count, dtype: int64
```

*Lưu ý: Đây là bước tạo nhãn tiềm ẩn **nhiễu**.*

CLIP – ZERO-SHOT CLASSIFICATION RESULT

```
Zero-Shot Accuracy: 0.6008. Macro-F1: 0.4757917338107088
              precision    recall  f1-score   support

        ball       0.29      0.77      0.42       957
        bike       0.27      0.91      0.42       574
    building       0.36      0.27      0.31      1567
         car       0.24      0.57      0.34       366
         dog       0.75      0.91      0.82      1806
         man       0.85      0.53      0.65     13042
        tree       0.28      0.16      0.20       699
       water       0.36      0.57      0.44      1096
       woman       0.63      0.75      0.69      6190

    accuracy                           0.60     26297
   macro avg       0.45      0.60      0.48     26297
weighted avg       0.69      0.60      0.61     26297
```

- **Accuracy = 0.60**: Mức hiệu năng trung bình, chấp nhận được cho baseline
- **Macro-F1 = 0.48**: Hiệu suất không đồng đều giữa các lớp.
- Có thể thấy mô hình có xu hướng hoạt động kém trên các lớp thiểu số / khó phân biệt.
- **Nguyên nhân chính**:
  - Weak labelling gây nhiễu và thiếu tính đại diện
  - CLIP zero-shot thiên về pattern phổ biến, chưa học rõ ranh giới phân lớp
  - Dữ liệu bị thiên lệch
- Kết luận: Phù hợp làm baseline nhanh.

CLIP – FEW-SHOT CLASSIFICATION RESULT

Few-Shot (128 shots) Accuracy: 0.7429. Macro-F1: 0.6399

	precision	recall	f1-score	support
ball	0.56	0.88	0.68	829
bike	0.37	0.94	0.53	446
building	0.44	0.79	0.56	1439
car	0.31	0.84	0.45	238
dog	0.84	0.96	0.90	1678
man	0.97	0.65	0.78	12914
tree	0.33	0.75	0.46	571
water	0.39	0.87	0.54	968
woman	0.89	0.82	0.85	6062
accuracy			0.74	25145
macro avg	0.57	0.83	0.64	25145
weighted avg	0.85	0.74	0.77	25145

- Accuracy (0.74), Macro-F1 (0.64) → Few-shot cải thiện đáng kể nhưng vẫn còn mất cân bằng giữa các lớp.
- Mô hình "trúng nhiều nhưng sai cũng nhiều" (Recall > Precision).
- Các lớp chủ thể động (*dog, woman, man*) đạt F1 cao nhờ đặc trưng thị giác rõ ràng và embedding CLIP tách biệt tốt.
- Các lớp tĩnh (*car, tree, bike*) dễ nhầm lẫn với bối cảnh phức tạp. Logistic Regression chưa đủ mạnh để phân tách các lớp khó này.
- Macro-F1 phản ánh thực chất hơn Weighted-avg F1 (0.77) vốn bị kéo lên do tập dữ liệu có số lượng mẫu lớn ở lớp man và woman.

HYPERPARAMETERS TUNING

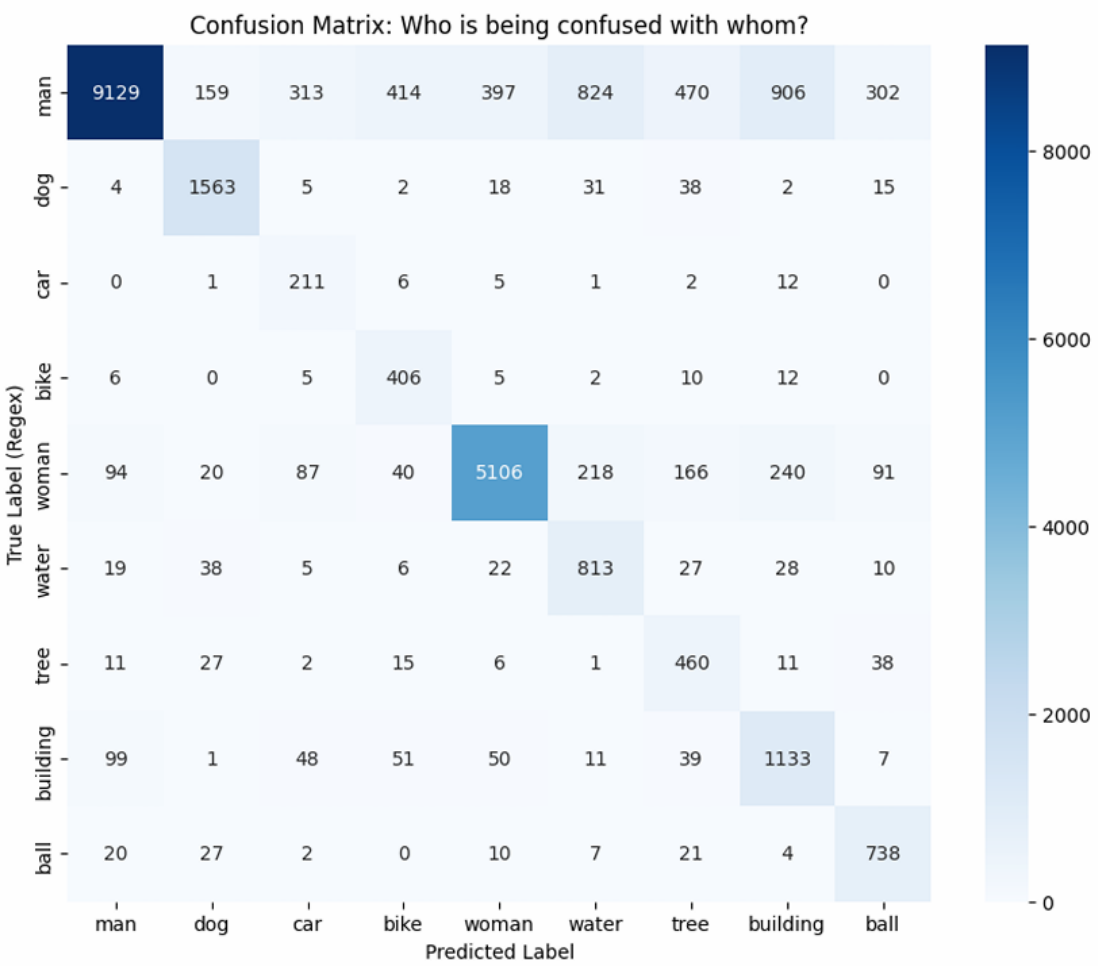
```python
# 1. Định nghĩa không gian tham số (Search Space)
param_grid = {
    'C': [0.01, 0.1, 1.0, 10.0, 100.0],  # Độ mạnh của
    # 'l1_ratio': [0] ,                  # penalty was depr
    'penalty': ['l2'],
    'class_weight': [None, 'balanced'],   # Xử lý mất c
    'solver': ['lbfgs', 'saga']          # Saga tốt hơn
}

# 2. Định nghĩa các hàm chấm điểm
scoring = {
    'Accuracy': 'accuracy',
    'Macro_F1': make_scorer(f1_score, average='macro')
}

# 3. Khởi tạo Grid Search với 5-Fold Cross Validation
grid_search = GridSearchCV(
    estimator=LogisticRegression(max_iter=1000),
    param_grid=param_grid,
    scoring=scoring,
    refit='Macro_F1',
    cv=5,
    verbose=1,
    n_jobs=-1
)
```

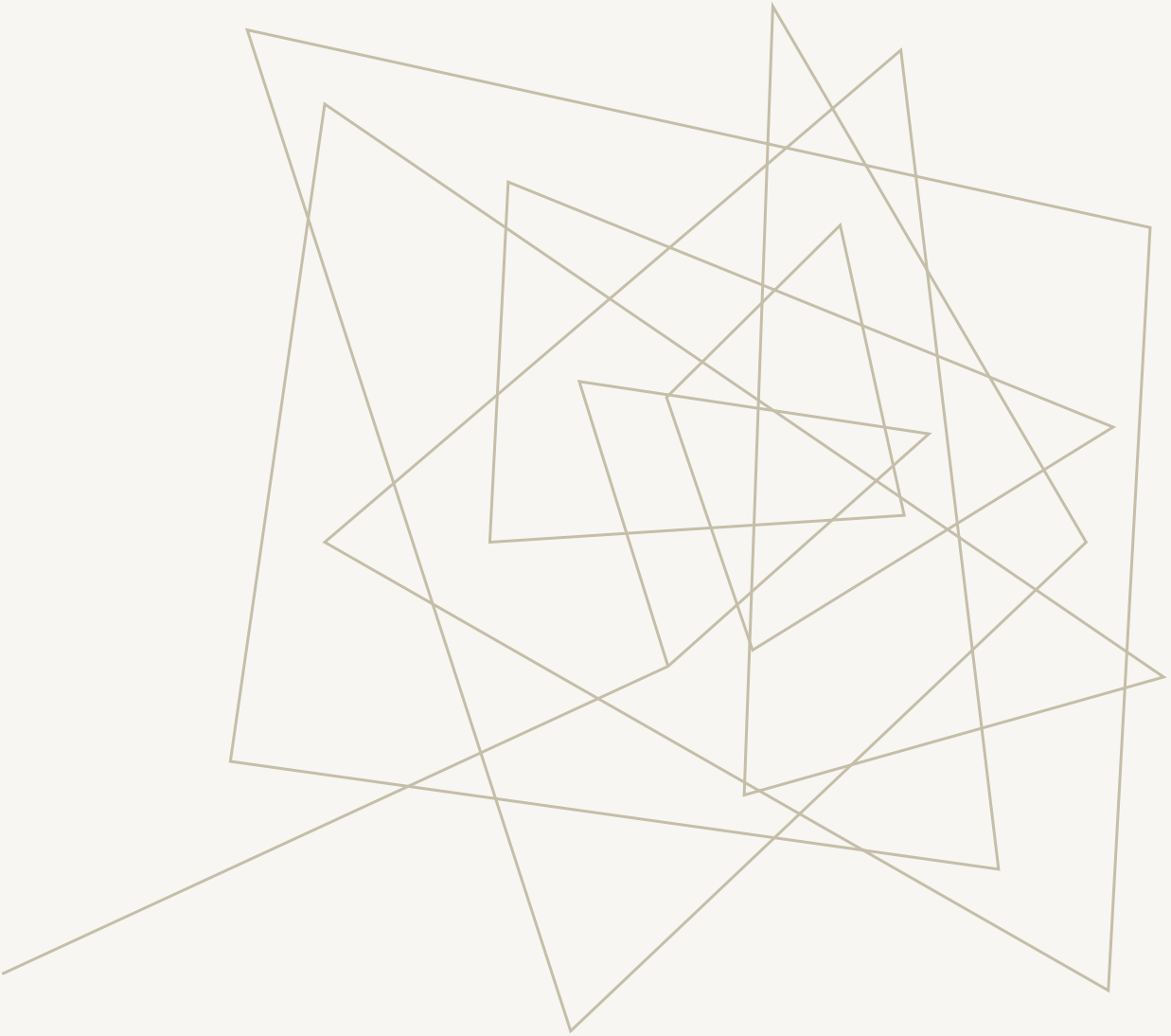
```
Bộ tham số tốt nhất (theo Macro-F1): {'C': 100.0, 'class_weight': None, 'penalty': 'l2', 'solver': 'saga'}
Macro-F1 cao nhất trên tập Train (CV): 0.8456
```

```python
fs_accuracy = accuracy_score(y_test, y_pred)
fs_f1_score = f1_score(y_test, y_pred, average='macro')
print(f"\nFew-Shot ({SHOTS_PER_CLASS} shots) Accuracy: {fs_accuracy:.4f}. Macro-F1: {fs_f1_score:.4f}")
print(classification_report(y_test, y_pred))
```

```
Few-Shot (128 shots) Accuracy: 0.7778. Macro-F1: 0.6700
              precision    recall  f1-score   support

        ball       0.61      0.89      0.73       829
        bike       0.43      0.91      0.59       446
    building       0.48      0.79      0.60      1439
         car       0.31      0.89      0.46       238
         dog       0.85      0.93      0.89      1678
         man       0.97      0.71      0.82     12914
        tree       0.37      0.81      0.51       571
       water       0.43      0.84      0.57       968
       woman       0.91      0.84      0.87      6062

    accuracy                           0.78     25145
   macro avg       0.60      0.84      0.67     25145
weighted avg       0.86      0.78      0.80     25145
```

ERROR ANALYSIS





THANK YOU!